

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Интеллектуальная система распознавания на основе изображений

лиц и отпечатков пальцев с интеграцией карт пропусков

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

А. В. Боев

(инициалы, фамилия)

Группа ПО-116

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2025 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

« ____ » _____ 20 ____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Боева А.В., шифр 21-06-0138, группа ПО-116

1. Тема «Интеллектуальная система распознавания на основе изображений лиц и отпечатков пальцев с интеграцией карт пропусков » утверждена приказом ректора ЮЗГУ от «04» апреля 2025 г. № 1696-с.

2. Срок предоставления работы к защите «9» июня 2025 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

- 1) проанализировать предметную область;
- 2) разработать концептуальную модель интеллектуальной системы распознавания;
- 3) спроектировать архитектуру программной системы распознавания;
- 4) реализовать и протестировать программную систему распознавания.

3.2. Входные данные и требуемые результаты для программы:

- 1) Входными данными для программной системы являются: изображения лиц, изображения отпечатков пальцев, изображения карт пропусков, предварительно сохранённые эталонные векторы признаков пользователей.

2) Выходными данными для программной системы являются: результат идентификации или верификации, сохранённые или обновлённые векторные представления.

4. Содержание работы (по разделам):

4.1. Введение.

4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.3. Технический проект: общие сведения о программной системе, описание используемых библиотек, проектирование архитектуры программной системы, проектирование пользовательского интерфейса программной системы.

4.4. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы, сборка компонентов программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цель и задачи работы.

Лист 3. Диаграмма прецедентов.

Лист 4. Архитектура системы.

Лист 5. Архитектура MTCNN.

Лист 6. Архитектура InceptionResnetV1.

Лист 7. Архитектура ResNet18.

Лист 8. Архитектура сиамской нейронной сети.

Лист 9. Метод получения изображения отпечатков пальцев.

Лист 10. Главные окна программы.

Лист 11. Заключение.

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

А. В. Боев

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 88 страницам. Работа содержит 23 иллюстрации, 23 таблицы, 20 библиографических источников и 11 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: биометрическая идентификация, нейронные сети, распознавание лиц, распознавание отпечатков пальцев, карта доступа, компьютерное зрение, интеллектуальная система, машинное обучение, обработка изображений, векторное представление.

Объект разработки: система распознавания пользователей на основе анализа изображений лиц и отпечатков пальцев с использованием карт доступа.

Цель выпускной квалификационной работы: разработка интеллектуальной системы, объединяющей методы компьютерного зрения и машинного обучения для распознавания лиц и отпечатков пальцев, интегрированной с картами доступа.

В процессе создания системы была разработана архитектура сиамской нейронной сети, создано настольное приложение с графическим интерфейсом для взаимодействия пользователей с системой, а также обучена разработанная нейронная сеть для распознавания отпечатков пальцев и протестированы MTCNN и InceptionResnetV1.

ABSTRACT

The volume of work is 88 pages. The work contains 23 illustrations, 23 tables, 20 bibliographic sources and 11 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The layout of the site, including the connection of components, is presented in annex B.

List of keywords: biometric identification, neural networks, face recognition, fingerprint recognition, access card, computer vision, intelligent system, machine learning, image processing, vector representation.

Object of development: user recognition system based on face and fingerprint image analysis using access cards.

The purpose of the final qualification work: development of an intelligent system combining computer vision and machine learning methods for face and fingerprint recognition, integrated with access cards.

In the process of creating the system, the architecture of the Siamese neural network was developed, a desktop application with a graphical interface for user interaction with the system was created, and the developed neural network for fingerprint recognition was trained and MTCNN and InceptionResnetV1 were tested.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	11
1 Анализ предметной области	14
1.1 Понятие распознавания лица	14
1.2 История развития систем распознавания лица	14
1.3 Выбор метода распознавания лица	18
1.3.1 Алгоритм Виолы-Джонса (Haar Cascades)	19
1.3.2 Гистограмма направленных градиентов (HOG)	19
1.3.3 Сверточные нейронные сети и MTCNN	20
1.4 Проблемы и недостатки современных методов распознавания	20
2 Техническое задание	23
2.1 Основание для разработки	23
2.2 Цель и назначение разработки	23
2.3 Требования к программной системе	23
2.3.1 Требования к данным программной системы	23
2.3.2 Функциональные требования к интеллектуальной системе	24
2.3.2.1 Сценарий использования «Добавление изображений лица»	25
2.3.2.2 Сценарий использования «Добавление отпечатков»	26
2.3.2.3 Сценарий использования «Создание карты доступа»	26
2.3.2.4 Сценарий использования «Сканирование карты»	27
2.3.2.5 Сценарий использования «Распознавание лица»	27
2.3.2.6 Сценарий использования «Сравнение отпечатков»	28
2.3.3 Требования пользователя к интерфейсу приложения	28
2.3.4 Нефункциональные требования к программной системе	29
2.3.4.1 Требования к надежности	29
2.3.4.2 Требования к программному обеспечению	29
2.3.4.3 Требования к аппаратному обеспечению	29
2.4 Требования к оформлению документации	30
3 Технический проект	31
3.1 Общая характеристика организации решения задачи	31

3.2	Обоснование выбора технологии проектирования	31
3.2.1	Язык программирования Python	31
3.2.2	Библиотека OpenCV	32
3.2.3	Библиотека PyTorch	32
3.2.4	Библиотека NumPy	33
3.2.5	Библиотека tkinter	33
3.2.6	Библиотека Python Imaging Library	34
3.3	Архитектура программной системы	34
3.4	Описание нейронных сетей	36
3.4.1	Архитектура MTCNN	36
3.4.2	Архитектура InceptionResnetV1	38
3.4.3	Архитектура ResNet18	40
3.4.4	Архитектура сиамской нейронной сети	42
3.4.4.1	Обучение сиамской нейронной сети	43
3.5	Проектирование пользовательского интерфейса	43
4	Рабочий проект	47
4.1	Модули программной системы	47
4.1.1	Модуль programm.py	47
4.1.2	Модуль save_face.py	50
4.1.3	Модуль embedding_create.py	51
4.1.4	Модуль func.py	52
4.1.5	Модуль fp.py	55
4.1.6	Модуль model.py	56
4.1.7	Модуль dataset.py	58
4.1.8	Модуль train.py	60
4.2	Модульное тестирование разработанного программного решения	60
4.3	Системное тестирование разработанного программного решения	62
4.4	Сборка программной системы	68
	ЗАКЛЮЧЕНИЕ	70
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	71
	ПРИЛОЖЕНИЕ А Представление графического материала	75

ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	87
На отдельных листах (CD-RW в прикрепленном конверте)	88
Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
Цель и задачи работы (Графический материал / Цель и задачи работы.png)	Лист 2
Диаграмма прецедентов (Графический материал / Диаграмма прецедентов.png)	Лист 3
Архитектура системы (Графический материал / Архитектура системы.png)	Лист 4
Архитектура MTCNN (Графический материал / Архитектура MTCNN.png)	Лист 5
Архитектура InceptionResnetV1 (Графический материал / Архитектура InceptionResnetV1.png)	Лист 6
Архитектура ResNet18 (Графический материал / Архитектура ResNet18.png)	Лист 7
Архитектура сиамской нейронной сети (Графический материал / Архитектура сиамской нейронной сети.png)	Лист 8
Метод получения изображения отпечатков пальцев (Графический материал / Метод получения изображения отпечатков пальцев.png)	Лист 9
Главные окна программы (Графический материал / Главные окна программы.png)	Лист 10
Заключение (Графический материал / Заключение.png)	Лист 11

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ИС – интеллектуальная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

ТЗ – техническое задание.

ТП – технический проект.

НС – нейронная сеть.

СНС – сверточная нейронная сеть.

CPU (Central Processing Unit) – центральный процессор.

GPU (Graphics Processing Unit) – специализированный микропроцессор, предназначенный для обработки графики и вывода изображений на экран.

MTCNN (Multi-task Cascaded Convolutional Neural Network) – алгоритм обнаружения лиц и определения их ключевых точек на изображениях.

UML (Unified Modelling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

ВВЕДЕНИЕ

Интеллектуальные системы распознавания на основе изображений стали неотъемлемой частью современных технологий, стремительно развиваясь с момента появления первых алгоритмов компьютерного зрения. Первые шаги в этой области были связаны с простыми задачами классификации и детекции объектов, однако сегодня такие системы способны решать сложные задачи: от медицинской диагностики до автономного управления транспортом. Интенсивность развития технологий обработки изображений не имеет аналогов — они трансформируют промышленность, медицину, безопасность и повседневную жизнь. Современные системы видеонаблюдения, беспилотные автомобили, автоматизированные производственные линии и даже социальные сети невозможно представить без алгоритмов машинного зрения.

Интеллектуальные системы распознавания основаны на методах глубокого обучения и нейронных сетей, которые позволяют анализировать изображения с высокой точностью, в отличие от традиционных алгоритмов, требующих ручной настройки признаков. Благодаря этому они способны адаптироваться к изменяющимся условиям, улучшая свою производительность с увеличением объема данных.

Современные компании, осознавая потенциал искусственного интеллекта, активно внедряют системы компьютерного зрения для оптимизации бизнес-процессов. Например, в розничной торговле такие технологии помогают анализировать поведение покупателей, в промышленности — контролировать качество продукции, а в медицине — автоматизировать диагностику заболеваний. Разработка интеллектуальных систем распознавания позволяет не только повысить эффективность работы предприятий, но и создать новые сервисы, привлекающие клиентов.

Сайт или мобильное приложение, использующее технологии компьютерного зрения, может стать мощным инструментом для привлечения аудитории. Например, системы дополненной реальности (AR) или сервисы распознавания объектов в реальном времени способны значительно улучшить

пользовательский опыт. Поэтому создание интеллектуальной системы на основе обработки изображений — это не только техническая задача, но и возможность вывести бизнес на новый уровень.

Цель настоящей работы – разработка интеллектуальной системы, объединяющей методы компьютерного зрения и машинного обучения для распознавания лиц и отпечатков пальцев, интегрированной с картами доступа. Для достижения поставленной цели необходимо решить *следующие задачи*:

- провести анализ предметной области и существующих решений;
- разработать концептуальную модель системы и выбрать методы распознавания;
- спроектировать архитектуру программной системы распознавания на основе изображений лиц и отпечатков пальцев;
- реализовать приложение, используя графический интерфейс.

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 88 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе на стадии анализа предметной области изучается история развития систем распознавания, сравниваются существующие технологии распознавания и их преимущества и недостатки.

Во втором разделе на стадии технического задания приводятся требования к разрабатываемой интеллектуальной системе распознавания.

В третьем разделе на стадии технического проектирования представлены проектные решения для системы распознавания.

В четвертом разделе приводится список классов и их методов, использованных при разработке системы распознавания, производится тестирование разработанного настольного приложения.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода.

1 Анализ предметной области

1.1 Понятие распознавания лица

Распознавание лиц — это технология, позволяющая установить личность человека на основе изображения, видеозаписи или других визуальных данных, связанных с лицом. Несмотря на большое разнообразие технических решений, все системы распознавания лиц стремятся к единой цели — точному сопоставлению биометрических данных с конкретным человеком из базы[1].

Процесс распознавания лица можно разделить на четыре основных этапа:

1. Обнаружение лица на изображении.
2. Определение ключевых точек лица.
3. Преобразование в цифровой вид.
4. Сопоставление полученной информации с базой данных.

1.2 История развития систем распознавания лица

Первые научные исследования в области распознавания лиц были начаты американским математиком Вудроу Бледсо в 1960-х годах. На этом первоначальном этапе процесс автоматизации был реализован лишь частично: человек самостоятельно отмечал ключевые точки лица. Ключевыми точками считались центры зрачков, уголки глаз, средняя точка на линии роста волос в верхней части лба и другие. Затем для этих ключевых точек вычислялся набор из двадцати взаимных расстояний, которые и служили формальным описанием лица.

Такая полуавтоматическая система демонстрировала производительность на уровне до 40 распознанных лиц в час, причем основным ограничивающим фактором выступала скорость работы человека, ответственного за корректную расстановку ключевых точек. Несмотря на низкую производительность по современным меркам, подобные системы отлично подходили

для задач идентификации преступников по фотографиям, где требования к скорости обработки были существенно ниже, чем в реальном времени.

В 1970-х годах были предприняты первые попытки полной автоматизации процесса расстановки ключевых точек, однако достигнутые результаты не отличались достаточной надежностью.

В 1987 г. был предложен подход, основанный на представлении изображения лица собственными векторами, полученными из матрицы изображения. Подход получил название *eigenface*. Основу этого подхода составляло представление черно-белого изображения лица в виде матрицы яркостей пикселей. После центрирования данных путем вычитания среднего значения яркости, матрица преобразовывалась в вектор, для которого строилась матрица ковариаций. Для этой матрицы вычислялся собственный вектор (*eigenvectors*), причем несколько наибольших по модулю компонент такого вектора использовались в качестве компактного описания лица. Изначально алгоритм изначально не был специфически разработан для задач распознавания лиц, а представлял собой общий метод анализа образов. Однако, следует отметить, что такое представление информации послужило основой для последующего развития более специализированных алгоритмов. В дальнейшем этот подход был адаптирован для выделения и анализа отдельных частей лица: глаз, носа и рта.

В 1990-х годах специально для решения задачи распознавания лиц были сформированы первые крупномасштабные базы данных лиц. Эти базы данных содержали тысячи изображений, сделанных в различных условиях освещения и с небольшими вариациями наклона головы. В отличие от ранее использовавшихся фотографий преступников, сделанных в стандартизированных условиях после задержания, новые базы данных позволяли более объективно оценивать качество алгоритмов распознавания. Например, база данных FERET включала более 14 000 изображений 1 200 различных людей и стала стандартным инструментом для сравнительного анализа методов распознавания. В 1990-х годах системы автоматического распознавания лиц начали находить первое коммерческое применение.

В 2001 г. появился быстрый алгоритм для решений задачи детекции – метод Виолы-Джонса. Этот метод основывался на использовании интегрального представления изображения, где значение каждого пикселя вычислялось как сумма значений всех пикселей, расположенных левее и выше данного.

Алгоритм реализует скользящее окно, которое последовательно движется по всему изображению с заданным шагом, вычисляя разность сумм пикселей внутри черных и белых областей специальных карт признаков (рис 1.1). Каждая такая карта признаков кодировала определенные характеристики частей лица. Процесс сканирования повторялся многократно с различными размерами окна и разными наборами карт признаков, что позволяло получить комплексное описание изображения в виде набора признаков Хаара. Для повышения эффективности в алгоритме Виолы-Джонса использовался каскадный принцип вычислений: если на начальных этапах обработки определенной области изображения не обнаруживалось достаточного количества соответствий, дальнейшие вычисления для этой области прекращались. Применение такой оптимизации метода позволило достичь высокой скорости обработки информации и получить широкое распространение алгоритма, в частности, в цифровых фотоаппаратах для реализации функции автоматического обнаружения лиц.

Необходимо отметить, что предложенный подход имеет и существенные ограничения: он способен надежно детектировать только фронтальные изображения лиц с наклоном не более 30 градусов и подвержен относительно высокому уровню ложных срабатываний.

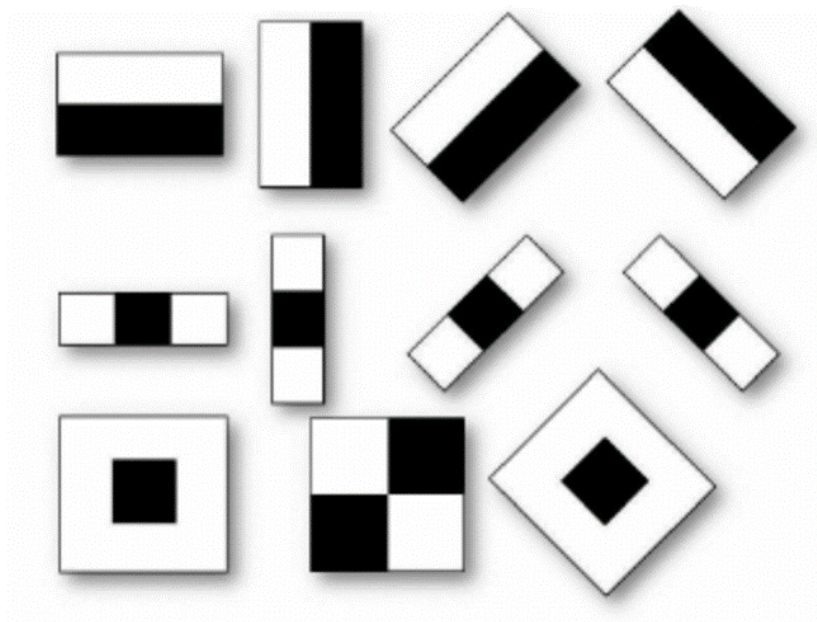


Рисунок 1.1 – Карта признаков Хаара

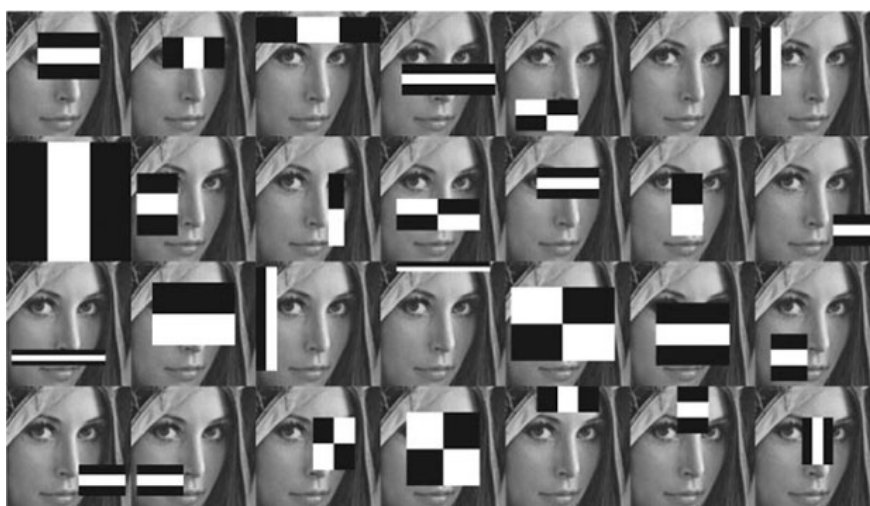


Рисунок 1.2 – Пример детекции частей лица картами признаков Хаара

С начала 2000-х гг. для распознавания лиц используется метод опорных векторов, скрытые марковские модели и начинают использоваться простые свёрточные нейронные сети.

В этот же период появились новые масштабные коллекции изображений лиц, такие как Labeled Faces in the Wild (LFW), содержащие 13 000 изображений 1 680 различных людей, сделанных в естественных условиях с вариациями освещения, выдержки и поз.

В 2007 году был предложен комбинированный подход, сочетающий локальные бинарные шаблоны (LBP) и фильтры Габора. LBP-преобразование эффективно сохраняло информацию о мелких деталях изображения, в то время как фильтры Габора обеспечивали инвариантность к масштабу и сохраняли информацию о общей форме лица. На заключительном этапе применялся метод главных компонент (PCA) для получения компактного описания лица.

Вместе с рядом прорывов в задачах классификации изображений (2012 г. – AlexNet) заметно улучшаются и системы распознавания лиц. Рост вычислительных мощностей, включая широкое использование GPU, сделал возможным обучение глубоких нейронных сетей, в частности свёрточных нейронных сетей (CNN).

Важным преимуществом CNN стало то, что они автоматически обучались оптимальным фильтрам для выделения значимых признаков, избавляя разработчиков от необходимости ручного подбора характеристик. Нейроны в различных слоях сети специализировались на распознавании признаков разного уровня абстракции: от простых геометрических примитивов (линий, углов) на начальных слоях до сложных семантических признаков (глаза, нос, рот) на глубоких слоях.

В 2014 году система DeepFace продемонстрировала точность распознавания 97% на базе данных LFW, впервые достигнув человеческого уровня.

Современные системы на основе глубокого обучения, такие как FaceNet и ArcFace, демонстрируют точность выше 99.9% на стандартных тестовых наборах данных. Они обладают устойчивостью к изменениям освещения, ракурса и частичным перекрытиям лица[2].

1.3 Выбор метода распознавания лица

В настоящее время создано множество методов распознавания лиц, различающихся по точности, скорости, устойчивости к внешним условиям и требованиям к ресурсам. Наиболее популярными являются следующие подходы: алгоритм Виолы-Джонса (Haar Cascades), метод гистограмм на-

правленных градиентов (HOG) и многозадачная сверточная нейронная сеть (MTCNN)[3].

1.3.1 Алгоритм Виолы-Джонса (Haar Cascades)

Алгоритм Виолы-Джонса — это один из первых и наиболее известных методов обнаружения объектов на изображениях в реальном времени, особенно лиц[4]. Он был представлен Полом Виолой и Майклом Джонсом в 2001 году и до сих пор используется в различных приложениях компьютерного зрения благодаря своей эффективности и скорости. Он основан на использовании простых признаков — так называемых признаков Хаара, напоминающих вейвлет-фильтры. Метод последовательно применяет каскад классификаторов, обученных с использованием алгоритма AdaBoost. Каждый этап каскада фильтрует изображения, уменьшая количество проверяемых областей.

Основное преимущество алгоритма — высокая скорость распознавания при малом потреблении ресурсов. Однако метод чувствителен к изменению условий: освещению, положению головы, мимике, а также плохо работает с тёмным оттенком кожи и при частичном перекрытии лица. Сегодня данный подход используется преимущественно в простых задачах с контролируемыми условиями.

1.3.2 Гистограмма направленных градиентов (HOG)

Гистограмма направленных градиентов (Histogram of Oriented Gradients) представляет изображение в виде вектора признаков, отражающего ориентацию градиентов яркости в отдельных ячейках изображения. Эти признаки являются устойчивыми к небольшим изменениям освещения и масштабирования. Далее полученный дескриптор подаётся в классификатор, как правило, линейный метод опорных векторов[5].

HOG был популярен в системах распознавания пешеходов и других объектов с устойчивыми очертаниями. Однако при распознавании лиц метод показывает невысокую точность. Он плохо справляется с поворотами головы

и изменениями выражения лица, поскольку чувствителен к геометрическим искажениям. Кроме того, метод имеет высокую вычислительную сложность: необходимо рассчитать градиенты для каждого пикселя, что замедляет обработку, особенно на слабых устройствах.

1.3.3 Сверточные нейронные сети и MTCNN

Современные системы распознавания лиц преимущественно основаны на нейросетевых архитектурах. Одним из таких решений является многозадачная сверточная нейронная сеть — MTCNN (Multi-task Cascaded Convolutional Neural Network)[6]. Данный подход объединяет детекцию лиц и определение ключевых точек (глаз, нос, рта) в рамках единой модели. MTCNN состоит из трёх последовательно работающих сетей: P-Net, R-Net и O-Net, каждая из которых уточняет результаты предыдущей.

Главное преимущество MTCNN — высокая точность и устойчивость к различным условиям: поворот головы, частичное перекрытие, различные оттенки кожи и освещения. Метод может эффективно работать даже с деформированными и профильными изображениями лиц. При наличии графического ускорителя (GPU) обработка осуществляется быстро, что делает этот подход оптимальным выбором для реальных систем биометрической идентификации.

1.4 Проблемы и недостатки современных методов распознавания

Современные системы распознавания лиц на основе искусственного интеллекта могут достигать точности более 95%, а некоторые системы достигают 99,5%[7]. Однако это возможно только в лабораторных или строго управляемых условиях: при хорошем освещении, правильном угле обзора, высоком разрешении камер и минимальных помехах. В реальных условиях эксплуатации — в общественных местах, на улице или в транспорте — точность существенно снижается, а риски ошибок возрастают.

На точность распознавания влияют следующие факторы:

1. Освещение и поза. Программы анализа лиц лучше всего работают с изображениями, на которых человек смотрит прямо в камеру. Условия освещения должны быть достаточно яркими, чтобы зафиксировать все черты лица, но при этом не засвечивать их. Положение лица также меняется при повороте головы и зависит от угла обзора наблюдателя.

2. Выражения лица. Нейтральное выражение идеально подходит для точного распознавания. Однако наши эмоции и настроение постоянно меняются, как и мимика. Эти различия меняют внешний вид лица, и для систем распознавания становится трудным его идентифицировать.

3. Старение. Естественные возрастные изменения внешности человека также влияют на способность систем распознавания лиц аутентифицировать людей.

4. Косметологические вмешательства, смена стиля. Изменения во внешности с помощью пластических операций, макияжа, смены причёски могут негативно отразиться на точности срабатывания алгоритмов распознавания.

5. Аксессуары. Наличие на лице различных объектов (очки, борода, медицинские маски и т.д.) или ношение других аксессуаров, таких как головные уборы, шарфы, может серьезно повлиять на работу системы распознавания. Но современные алгоритмы отрабатывают способность видеть сквозь эти препятствия[7].

В условиях, отличных от идеальных, такие системы все ещё способны допускать серьёзные ошибки. За всю историю развития распознавания лиц в России и мире происходило много крупных скандалов, связанных с использованием автоматических систем биометрической идентификации.

Больше всего скандалов случается, когда системы распознавания лиц путают людей с похожими на них преступниками, вследствие чего людям приходится доказывать свою непричастность перед правоохранительными органами. Один из наиболее известных инцидентов произошёл в Москве, где система распознавания лиц, интегрированная в камеры видеонаблюдения, неверно определила личность прохожего, что привело к его задержанию.

Впоследствии выяснилось, что ошибка была вызвана значительным визуальным сходством с разыскиваемым человеком и недостаточной уникальностью биометрических признаков[8]. В подобных случаях ошибочное распознавание лиц является неприемлимым, а 5% ошибок - слишком большим числом.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Интеллектуальная система распознавания на основе изображений лица и отпечатков пальцев с интеграцией карт пропусков».

2.2 Цель и назначение разработки

Интеллектуальная система предназначена для автоматизированного распознавания личности с использованием изображений лица, отпечатков пальцев и пропускных карт. Система разрабатывается для применения в задачах контроля доступа на охраняемые объекты, в учреждениях с ограниченным доступом, а также в корпоративной среде.

Система должна иметь возможность проведения идентификации посредством встроенного модуля биометрической аутентификации, без необходимости ввода паролей или взаимодействия с охранным персоналом[9].

Задачами данной разработки являются:

- реализация распознавания на основе изображений лица;
- реализация распознавания на основе изображений отпечатков пальцев;
- интеграция карт пропуска;
- реализация системы идентификации.

2.3 Требования к программной системе

2.3.1 Требования к данным программной системы

Для обучения нейронных сетей и анализа изображений лиц и отпечатков пальцев программной системе требуется датасет, состоящий из изображений в формате JPEG или PNG, сохранённых в отдельных папках по типу

биометрических данных. Каждое изображение должно соответствовать одному пользователю и быть предварительно размечено[10].

Кроме того, для сохранения параметров обученных моделей нейросетей (модели для лиц и модели для отпечатков) используются файлы формата .pth, содержащие веса и смещения соответствующих моделей[11].

Для анализа новых изображений и последующего сравнения с эталонными эмбедингами также требуются файлы формата .pth, содержащие параметры нейронных сетей, полученные в процессе предварительного обучения.

Эмбединги, извлекаемые из изображений, сохраняются в формате .npy или .pkl и используются для сравнения биометрических признаков при распознавании личности. Все данные организованы по структуре, обеспечивающей однозначную связь между пользователем, изображениями, эмбедингами и идентификатором карты доступа.

2.3.2 Функциональные требования к интеллектуальной системе

В разрабатываемой интеллектуальной системе должны быть реализованы следующие функции:

- добавление пользователя в систему;
- добавление изображения лица пользователя;
- добавление изображения отпечатка пальца пользователя;
- создание карты доступа;
- распознавание пользователя;
- сканирование карты пользователя;
- распознавание лица;
- сравнение отпечатков.

На рисунке 2.1 предоставлены функциональные требования к системе, представленные в виде диаграммы прецедентов[12].

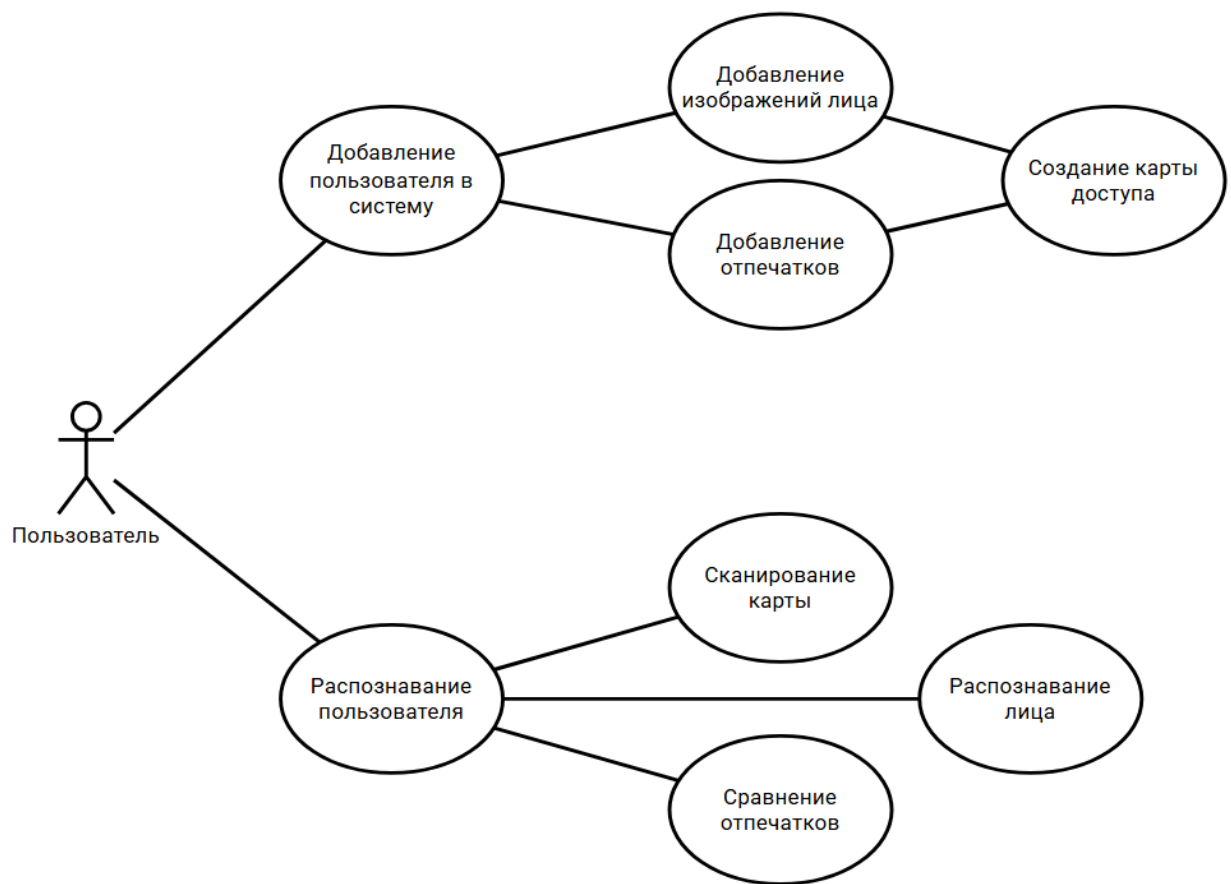


Рисунок 2.1 – Диаграмма прецедентов

2.3.2.1 Сценарий использования «Добавление изображений лица»

Заинтересованные лица и их требования: пользователь желает добавить изображения лица в систему.

Предусловие: программа запущена, выбран режим «Добавить в систему».

Постусловие: программа сохраняет изображения лица пользователя в систему.

Основной успешный сценарий:

1. Пользователь нажимает на кнопку «Добавить в систему».
2. Программа открывает диалоговое окно с указаниями для пользователя.
3. Пользователь закрывает диалоговое окно.
4. Программа запускает сканирование лица пользователя.

5. Пользователь выполняет указания программы.
6. Программа сохраняет изображения пользователя для дальнейшей обработки.
7. Программа открывает диалоговое окно с результатом работы.
8. Пользователь закрывает диалоговое окно и завершает сканирование лица.

2.3.2.2 Сценарий использования «Добавление отпечатков»

Заинтересованные лица и их требования: пользователь желает добавить изображения отпечатка в систему.

Предусловие: программа запущена, выбран режим «Добавить в систему», изображения лица добавлены.

Постусловие: программа сохраняет изображения отпечатка пользователя в систему.

Основной успешный сценарий:

1. Пользователь нажимает на кнопку «Добавить в систему».
2. Программа открывает диалоговое окно выбора файла.
3. Пользователь выбирает файл в формате .jpg, содержащий изображение отпечатка пальца.
4. Программа запускает обучение нейронной сети на основе изображения отпечатка.
5. Программа сохраняет модель нейронной сети для дальнейшей работы.

2.3.2.3 Сценарий использования «Создание карты доступа»

Заинтересованные лица и их требования: пользователь желает создать карту для доступа в систему.

Предусловие: программа запущена, выбран режим «Добавить в систему», изображения лица и отпечатка добавлены.

Постусловие: программа создает карту доступа в систему.

Основной успешный сценарий:

1. Пользователь нажимает на кнопку «Добавить в систему».
2. Программа запускает создание карты доступа.
3. Программа сохраняет карту доступа в систему.

2.3.2.4 Сценарий использования «Сканирование карты»

Заинтересованные лица и их требования: пользователь желает получить доступ в систему.

Предусловие: программа запущена, выбран режим «Распознавание пользователя».

Постусловие: программа переходит к распознаванию лица.

Основной успешный сценарий:

1. Пользователь нажимает на кнопку «Распознавание пользователя».
2. Программа открывает диалоговое окно с указаниями для пользователя.
3. Пользователь закрывает диалоговое окно.
4. Пользователь выполняет указания программы.
5. Программа запускает сканирование карты пользователя.
6. Программа получает данные с карты для дальнейшего распознавания.

2.3.2.5 Сценарий использования «Распознавание лица»

Заинтересованные лица и их требования: пользователь желает получить доступ в систему.

Предусловие: программа запущена, выбран режим «Распознавание пользователя», карта доступа распознана.

Постусловие: программа переходит к сравнению отпечатка.

Основной успешный сценарий:

1. Пользователь нажимает на кнопку «Распознавание пользователя».
2. Программа открывает диалоговое окно с указаниями для пользователя.
3. Пользователь закрывает диалоговое окно.

4. Пользователь выполняет указания программы.
5. Программа запускает сканирование карты пользователя.
6. Программа получает данные с карты для дальнейшего распознавания.

2.3.2.6 Сценарий использования «Сравнение отпечатков»

Заинтересованные лица и их требования: пользователь желает получить доступ в систему.

Предусловие: программа запущена, выбран режим «Распознавание пользователя», карта доступа и лицо распознаны.

Постусловие: программа выдает результат распознавания.

Основной успешный сценарий:

1. Пользователь нажимает на кнопку «Распознавание пользователя».
2. Программа открывает диалоговое окно с указаниями для пользователя.
3. Пользователь закрывает диалоговое окно.
4. Пользователь выполняет указания программы.
5. Программа запускает сканирование карты пользователя.
6. Программа получает данные с карты для дальнейшего распознавания.

2.3.3 Требования пользователя к интерфейсу приложения

Приложение должно иметь следующие экраны:

1. Экран «Идентификация». Основной экран, реализующий функционал распознавания пользователя на основе изображений лица и отпечатка пальца.
2. Экран «Добавление в систему». Экран, позволяющий добавлять пользователя в систему, путём сохранения изображений лица и отпечатков пальца с генерацией карты доступа.

2.3.4 Нефункциональные требования к программной системе

2.3.4.1 Требования к надежности

Программная система должна обеспечивать стабильную работу в различных условиях эксплуатации. В процессе работы приложения могут возникнуть следующие аварийные ситуации:

1. Отсутствие подключения к камере для получения изображений.
2. Ошибка доступа к файлам изображения.

Для предотвращения аварийных ситуаций программа должна корректно обрабатывать исключения при работе с файлами, предоставляя пользователям информативные сообщения об ошибках. В случае проблем с отсутствием прав доступа к директории сохранения файлов, полученных в результате работы программы, программа должна открывать диалоговое окно с выбором другой директории.

2.3.4.2 Требования к программному обеспечению

Для запуска и работы программы требуется компьютер под управлением операционной системы Windows 10 или Windows 11. При использовании графических адаптеров NVIDIA с поддержкой технологии CUDA для ускорения обучения нейронной сети необходима последняя версия драйверов соответствующего адаптера, а также программы CUDA Toolkit и cuDNN

2.3.4.3 Требования к аппаратному обеспечению

Для корректной работы программного продукта, реализующего распознавание лиц, отпечатков пальцев и идентификаторов карт доступа, требуется центральный процессор с количеством ядер от 6 и выше и тактовой частотой не менее 2.4 ГГц. Объем оперативной памяти должен составлять не менее 8 ГБ.

Для захвата изображений лиц обязательно наличие веб-камеры с разрешением не менее 1280×720 пикселей. Камера должна обеспечивать чёткое изображение при нормальном освещении.

Для сохранения изображений отпечатков пальцев может потребоваться сканер с разрешением не менее 500 dpi, совместимый с операционной системой и поддерживаемый программной частью.

2.4 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

Программная документация должна включать в себя:

- анализ предметной области;
- техническое задания;
- технический проект;
- рабочий проект.

3 Технический проект

3.1 Общая характеристика организации решения задачи

Необходимо спроектировать и разработать систему, которая должна выполнять распознавание на основе изображений лиц и отпечатков пальцев с использованием карт пропуска.

Система представляет собой программу, позволяющую пользователю выполнять распознавание лиц и отпечатков пальцев и сканирование карты пропуска для авторизации. Интерфейс включает в себя окно, отображающее видеопоток с камеры и результаты распознавания лица или отпечатка пальца и карты пропуска. Программа автоматически определяет лицо, отпечаток пальца или карту пропуска в кадре, сопоставляет данные и отображает результаты выполнения программы на экране.

3.2 Обоснование выбора технологии проектирования

Используемые для создания интеллектуальной системы языки программирования и технологии соответствуют современным практикам разработки, обеспечивают высокую производительность и отказоустойчивость системы.

3.2.1 Язык программирования Python

Язык программирования Python представляет собой высокоуровневый язык общего назначения, отличающийся лаконичным синтаксисом и высокой читаемостью кода[13]. Python обеспечивает возможность реализации объектно-ориентированного, функционального и процедурного стилей программирования[14]. Он обладает мощной стандартной библиотекой, которая охватывает множество областей применения — от работы с файлами и сетями до многопоточности и регулярных выражений[15]. Python широко используется в научных вычислениях, благодаря таким библиотекам как NumPy, SciPy, Pandas и Matplotlib. В области машинного обучения и нейросетей Python является наилучшим языком программирования, благодаря та-

ким инструментам как PyTorch, TensorFlow и Scikit-learn. Кроме того, язык поддерживается большим сообществом, что облегчает поиск решений и развитие проектов[16].

3.2.2 Библиотека OpenCV

OpenCV представляет собой мощную библиотеку компьютерного зрения и обработки изображений, предназначенную для выполнения широкого спектра задач, связанных с анализом изображений и видео. Она предоставляет высокоуровневые и низкоуровневые средства для работы с изображениями, включая функции для обработки, фильтрации, распознавания объектов и анализа движения. Основное преимущество OpenCV заключается в её высокой производительности, гибкости и поддержке широкого спектра алгоритмов компьютерного зрения. Благодаря интеграции с языком Python, OpenCV позволяет быстро разрабатывать приложения, связанные с обработкой изображений и видео, и является популярным инструментом как для учебных, так и для полноценных исследовательских и промышленных проектов.

3.2.3 Библиотека PyTorch

PyTorch — это современная библиотека машинного и глубокого обучения, предназначенная для создания и обучения нейронных сетей. Она предоставляет интуитивно понятные инструменты для работы с тензорами, автоматического дифференцирования и построения моделей. Одним из ключевых преимуществ PyTorch является использование динамической вычислительной графики, что делает разработку и отладку нейросетей более гибкой и наглядной. Библиотека поддерживает ускорение вычислений с помощью графических процессоров (GPU), что существенно повышает производительность при обучении моделей. Благодаря тесной интеграции с Python, а также множеству встроенных модулей и готовых архитектур, PyTorch широко применяется в задачах компьютерного зрения, обработки естественного языка, биометрии и других областях искусственного интеллекта[17].

3.2.4 Библиотека NumPy

NumPy представляет собой фундаментальную библиотеку для численных вычислений в Python, обеспечивающую поддержку многомерных массивов и высокоуровневых математических операций над ними. Благодаря высокоэффективным операциям с массивами и встроенным линейным алгебраическим методам, NumPy позволяет обрабатывать изображения больших размеров с минимальными затратами ресурсов, что делает её незаменимой в задачах, связанных с сжатием, декомпрессией и преобразованием графических данных.

3.2.5 Библиотека tkinter

Для построения графического интерфейса в проекте выбрана библиотека tkinter, которая является частью стандартной поставки Python и представляет собой обёртку над библиотекой Tcl/Tk. Этот выбор объясняется рядом технологических и практических преимуществ:

1. Отсутствие необходимости в установке сторонних зависимостей делает tkinter особенно удобным для использования в проектах с упрощённым процессом развёртывания.
2. Простота проектирования интерфейса позволяет легко создавать окна, формы, кнопки, меню, вкладки и другие элементы без необходимости изучения сложных графических фреймворков. Это способствует быстрому созданию прототипов и конечных версий интерфейса.
3. Полная кроссплатформенность интерфейса гарантирует его одинаковое отображение и поведение на всех операционных системах, что избавляет разработчика от необходимости писать отдельные реализации под разные платформы.
4. Наличие компонентов высокого уровня, таких как текстовые поля, выпадающие списки, кнопки, панели и таблицы, даёт возможность создать функциональный и интуитивно понятный интерфейс для работы с данными.

5. Возможность динамического обновления интерфейса обеспечивает удобную визуализацию изменений.

tkinter полностью удовлетворяет требованиям к простому, лёгкому в использовании, но функциональному графическому интерфейсу, особенно в условиях ограниченного времени и ресурсов.

3.2.6 Библиотека Python Imaging Library

Python Imaging Library, сокращенно PIL — это библиотека для работы с растровыми изображениями, обеспечивающая поддержку различных форматов, таких как JPEG, PNG, BMP и других. PIL предоставляет широкий набор функций для загрузки, сохранения, обработки и преобразования изображений, что упрощает различные взаимодействия с файлами изображений, такие как сохранение и загрузка изображений, поэтому она является удобным инструментом для предварительной и финальной обработки графических данных.

3.3 Архитектура программной системы

На рисунке 3.1 в виде UML-диаграммы представлена архитектура программной системы.

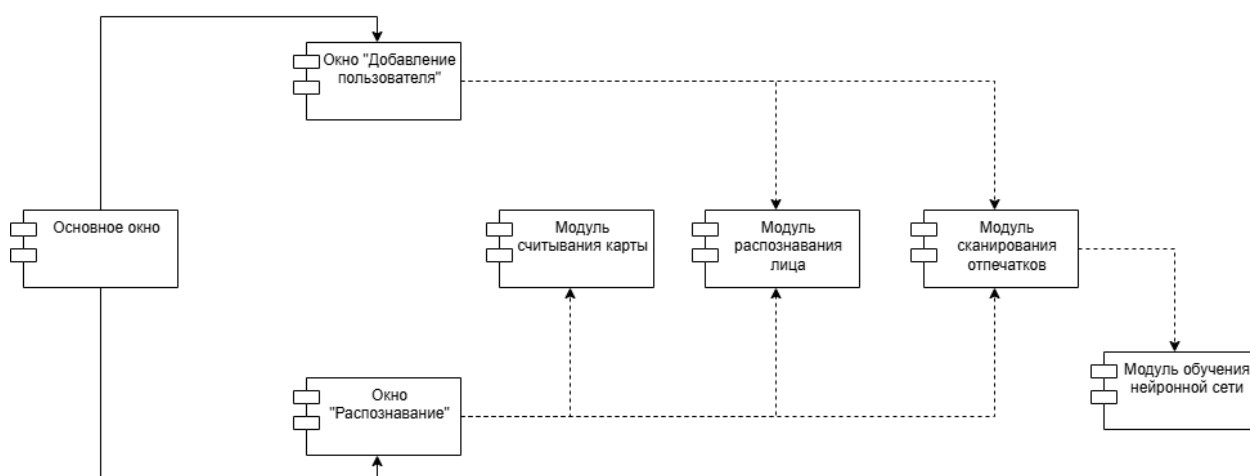


Рисунок 3.1 – Архитектура системы

Интеллектуальная система состоит из следующих компонентов:

1. Основное окно. Компонент «Основное окно» представляет собой центральный элемент системы, обеспечивающий взаимодействие с пользователем через графический интерфейс. Он отвечает за инициализацию и координацию работы всех подсистем, включая управление потоками и вызовы других модулей для выполнения задач, таких как добавление пользователя или распознавание.

2. Окно распознавания. Компонент «Окно распознавания» представляет собой специализированный интерфейс, отображающий процесс распознавания пользователя. Он взаимодействует с модулями распознавания лица и сканирования отпечатков, предоставляя пользователю визуальную обратную связь о результатах идентификации.

3. Окно добавления пользователя. Компонент «Окно добавления пользователя» реализует логику добавления нового пользователя в систему. Он управляет процессом сбора биометрических данных (лица и отпечатков пальцев) и их последующим сохранением в базе данных, обеспечивая интеграцию с модулями захвата и обработки данных.

4. Модуль считывания карты. Компонент «Модуль считывания карты» отвечает за обнаружение и интерпретацию карт, содержащих идентификационные данные пользователя. Он взаимодействует с камерой для считывания визуальной информации и передает данные в систему для дальнейшей верификации.

5. Модуль распознавания лица. Компонент «Модуль распознавания лица» предназначен для идентификации пользователя на основе анализа изображений лица, полученных с камеры. Он использует предварительно обученные модели для извлечения векторов и сравнения их с базой данных, обеспечивая точное распознавание.

6. Модуль сканирования отпечатков. Компонент «Модуль сканирования отпечатков» реализует функциональность сравнения отпечатков пальцев пользователя с сохраненными в базе данных образцами. Он применяет нейронные сети для анализа биометрических данных и определения степени сходства, что используется в процессе верификации.

7. Модуль обучения нейронной сети. Компонент «Модуль обучения нейронной сети» отвечает за обучение моделей, используемых для обработки биометрических данных, таких как лица и отпечатки пальцев. Он обеспечивает подготовку и настройку нейронных сетей.

3.4 Описание нейронных сетей

Для реализации системы распознавания были использованы три различные архитектуры нейронных сетей[18]. Для обнаружения лиц на изображении была использована модель MTCNN, для извлечения векторных признаков лица - InceptionResnetV1. Для распознавания отпечатков пальцев была использована сиамская нейронная сеть.

3.4.1 Архитектура MTCNN

Нейронная сеть MTCNN представляет собой каскадную систему для обнаружения лиц, состоящую из трёх последовательно работающих сетей: P-Net (сеть предложений), R-Net (сеть уточнения) и O-Net (выходная сеть).

На рисунке 3.2 изображена структура сети MTCNN.

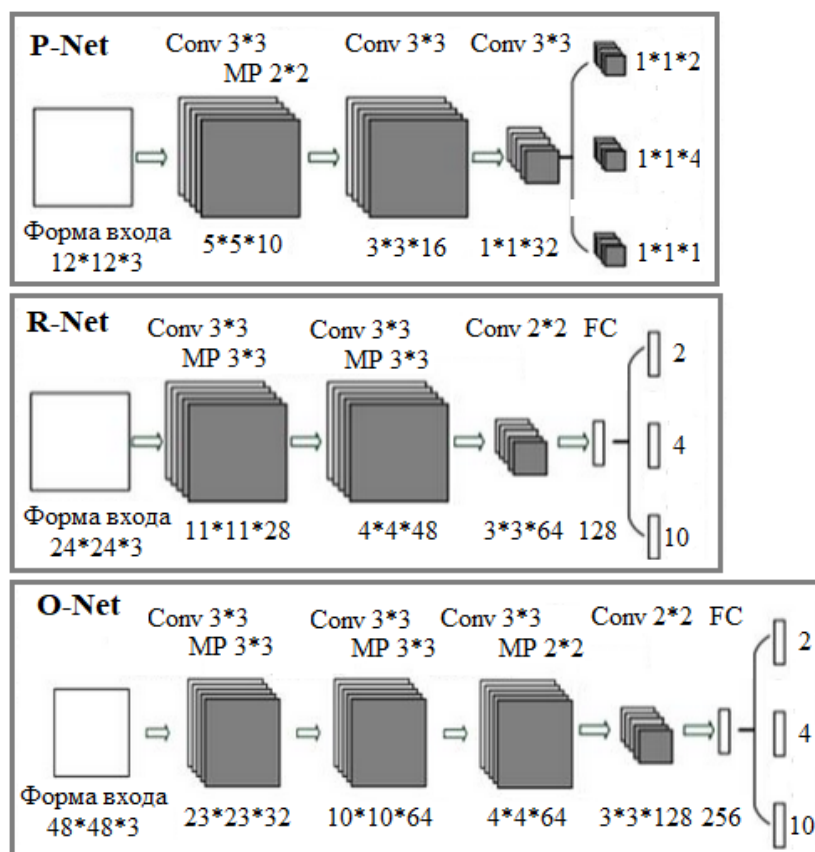


Рисунок 3.2 – Структура MTCNN

P-Net — это компактная сеть, которая быстро сканирует изображение и находит возможные места, где могут быть лица. Она работает с изображением, уменьшенным в несколько раз, чтобы учитывать лица разных размеров. Структура сети включает несколько слоёв обработки: сначала изображение проходит через фильтры, выделяющие основные признаки, затем уменьшается в размере, после чего ещё два набора фильтров уточняют анализ. В итоге P-Net выдаёт три результата: вероятность того, что в области есть лицо, координаты рамки вокруг лица и расположение ключевых точек, таких как глаза или рот. Однако эта сеть часто предлагает слишком много вариантов, включая ошибочные, поэтому лишние области отсеиваются специальным методом.

R-Net берёт области, предложенные P-Net, и проверяет их более тщательно, отбрасывая те, где лиц нет, и улучшая рамки вокруг настоящих лиц. Эта сеть сложнее, так как включает дополнительный слой, который объединяет все данные. Она начинается с фильтров, которые выделяют детали, затем

уменьшает изображение и применяет ещё несколько фильтров. После этого данные преобразуются в единый набор чисел и анализируются, чтобы определить, есть ли лицо, где оно находится и где расположены ключевые точки. Лишние рамки снова убираются, но с более строгим подходом.

O-Net — финальная сеть, которая делает последний и самый точный анализ. Она работает с более крупными участками изображения, чтобы рассмотреть лицо в деталях. Структура O-Net похожа на R-Net, но включает больше слоёв для обработки данных, что позволяет добиться высокой точности. Она выдаёт окончательные рамки вокруг лиц, вероятность их наличия и координаты ключевых точек.

Общее число параметров MTCNN составляет около 1–2 миллионов, с распределением примерно 10–20 тысяч для P-Net, 100–200 тысяч для R-Net и 300–500 тысяч для O-Net. Сеть предобучена на наборе данных WIDER FACE и используется в системе без дообучения. MTCNN обрабатывает кадры RGB, преобразованные из формата BGR, на устройстве CUDA или CPU.

3.4.2 Архитектура InceptionResnetV1

InceptionResnetV1 представляет собой гибридную архитектуру, объединяющую Inception-модули, которые параллельно применяют свёртки разных размеров для извлечения разнообразных признаков, и остаточные соединения, ускоряющие обучение и улучшающие сходимость. Сеть состоит из нескольких ключевых компонентов: начального блока (Stem), Inception-Resnet блоков (A, B, C), блоков уменьшения размерности (Reduction A, B) и финального слоя для создания эмбединга.

На рисунке 3.3 изображена структура сети InceptionResnetV1.

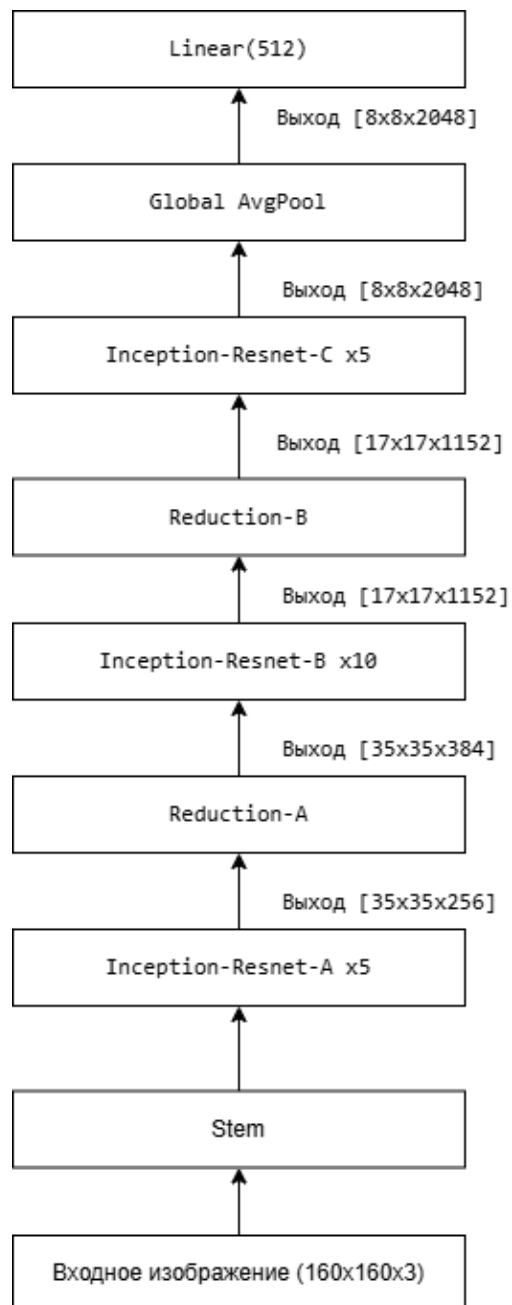


Рисунок 3.3 – Структура сети InceptionResnetV1

Работа сети включает в себя следующие этапы:

1. Предобработка: Изображение лица, вырезанное с помощью MTCNN, масштабируется до 160x160 и нормализуется.

2. Извлечение признаков: Stem-блок извлекает низкоуровневые признаки (края, текстуры), которые затем обрабатываются Inception-Resnet блоками. Inception-модули параллельно применяют свёртки разных размеров (1x1, 3x3, 7x1 и т.д.), захватывая признаки на разных масштабах. Остаточные соединения ускоряют обучение, добавляя вход блока к его выходу.

3. Уменьшение размерности: Reduction блоки сокращают пространственные размеры, увеличивая глубину признаков, что позволяет сети фокусироваться на высокоуровневых характеристиках (форма лица, черты).

4. Генерация векторного представления: Финальный пулинг и полносвязный слой преобразуют признаки в вектор 512D, который используется для сравнения лиц.

Векторные представления сравниваются по косинусному сходству, что определяет, принадлежат ли два изображения одному человеку.

3.4.3 Архитектура ResNet18

ResNet18 состоит из начального свёрточного слоя, четырёх блоков остаточных слоёв, глобального усредняющего пулинга и финального слоя.

На рисунке 3.4 изображена структура сети ResNet18.

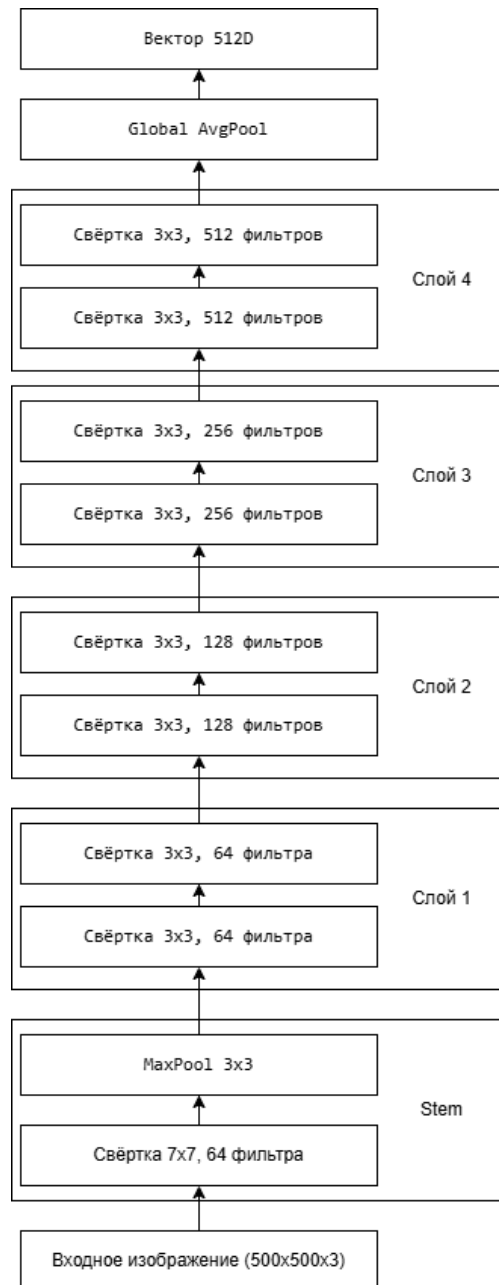


Рисунок 3.4 – Структура сети ResNet18

ResNet18 извлекает признаки из изображения отпечатка пальца, преобразуя его в компактный вектор, который кодирует уникальные характеристики узоров. Процесс включает следующие этапы:

1. Предобработка: Изображение отпечатка преобразуется в трёхканальное 500x500 RGB и нормализуется.
2. Извлечение признаков: Начальный блок извлекает низкоуровневые признаки (края, текстуры). Остаточные блоки (слои 1–4) постепенно увели-

чивают глубину признаков, уменьшая пространственные размеры. Остаточные соединения добавляют вход модуля к его выходу, упрощая обучение.

3. Генерация эмбединга: После глобального пулинга сеть возвращает вектор 512D, используемый в сиамской нейронной сети.

3.4.4 Архитектура сиамской нейронной сети

Сиамская нейронная сеть — это архитектура глубокого обучения, разработанная для задач сравнения пар объектов, таких как изображения, с целью определения их сходства или различия. Её ключевая особенность заключается в использовании двух идентичных подсетей с общими весами, которые обрабатывают пару входных данных и создают компактные представления (векторы), сравниваемые с помощью метрики сходства[19].

На рисунке 3.5 изображена структура сиамской нейронной сети.

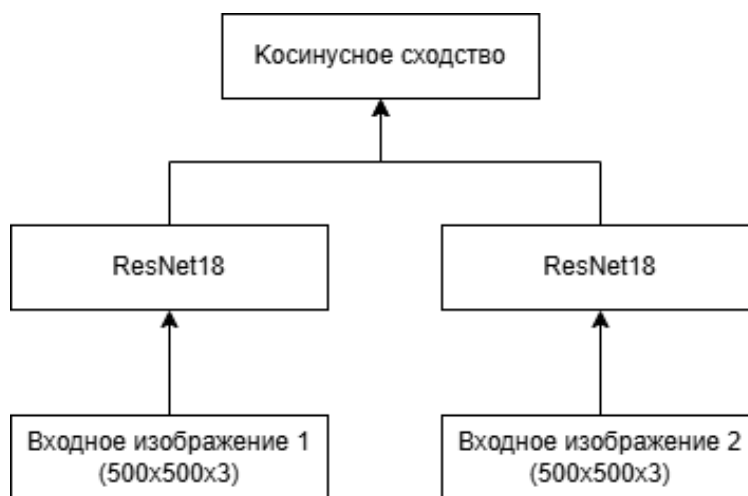


Рисунок 3.5 – Структура сиамской нейронной сети

Косинусное сходство вычисляется как

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|},$$

где A, B — векторы 512D. Значение сходства находится в диапазоне [0, 1], и для принятия решения используется порог: если сходство превышает порог, отпечатки считаются принадлежащими одному человеку.

3.4.4.1 Обучение сиамской нейронной сети

Обучение сиамской нейронной сети проводится на наборе изображений отпечатков пальцев формата JPEG или PNG разрешением 500×500 . Для создания обучающего набора формируются пары изображений: положительные пары включают комбинации отпечатков одного человека с меткой 1, а отрицательные пары состоят из случайных комбинаций отпечатка этого человека и отпечатка другого человека, с меткой 0. Изображения преобразуются в оттенки серого, масштабируются до размера 500×500 и конвертируются в тензоры для обработки сетью.

Обучение проводится в течение 20 эпох на группах из восьми пар изображений. Дополнительные аугментации изображений не применяются. Процесс обучения осуществляется на центральном процессоре, но предусмотрена возможность использования графического процессора при наличии подходящего оборудования. На каждой эпохе для каждой группы пар изображений входные данные и их метки отправляются на вычислительное устройство. Модель вычисляет вероятность сходства пары, после чего рассчитывается потеря по формуле бинарной кросс-энтропии:

$$L = -[y \cdot \log(p) + (1 - y) \cdot \log(1 - p)],$$

где y — истинная метка (1 для положительных пар, 0 для отрицательных), p — предсказанная вероятность сходства. Далее выполняется обратное распространение ошибки и обновление весов модели. После завершения обучения веса модели сохраняются в файл для последующего использования.

3.5 Проектирование пользовательского интерфейса

На основании требований к пользовательскому интерфейсу, представленных в пункте 2.3.3, был разработан графический интерфейс программной системы[20].

На рисунке 3.6 представлен макет интерфейса главного окна. Макет содержит следующие элементы:

1. Поле для отображения названия программы.

2. Кнопка перехода к окну "Добавление пользователя".
3. Кнопка перехода к окну "Распознавание".
4. Кнопка выхода из программы.

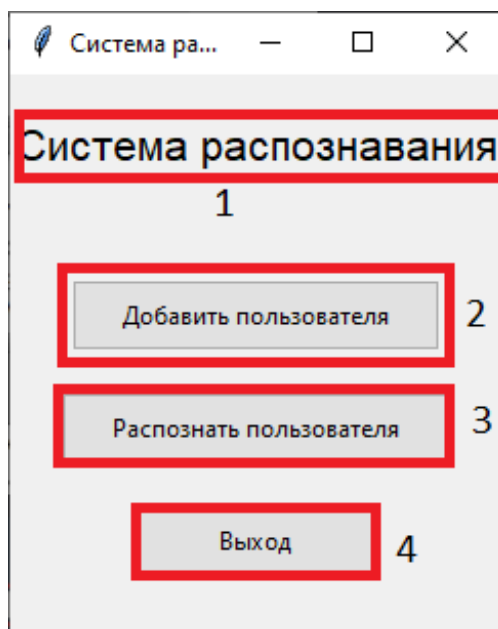


Рисунок 3.6 – Макет основного окна

На рисунке 3.7 представлен макет интерфейса окна "Распознавание".

Макет содержит следующие элементы:

1. Поле для отображения вывода с камеры.
2. Кнопка перехода к главному окну.

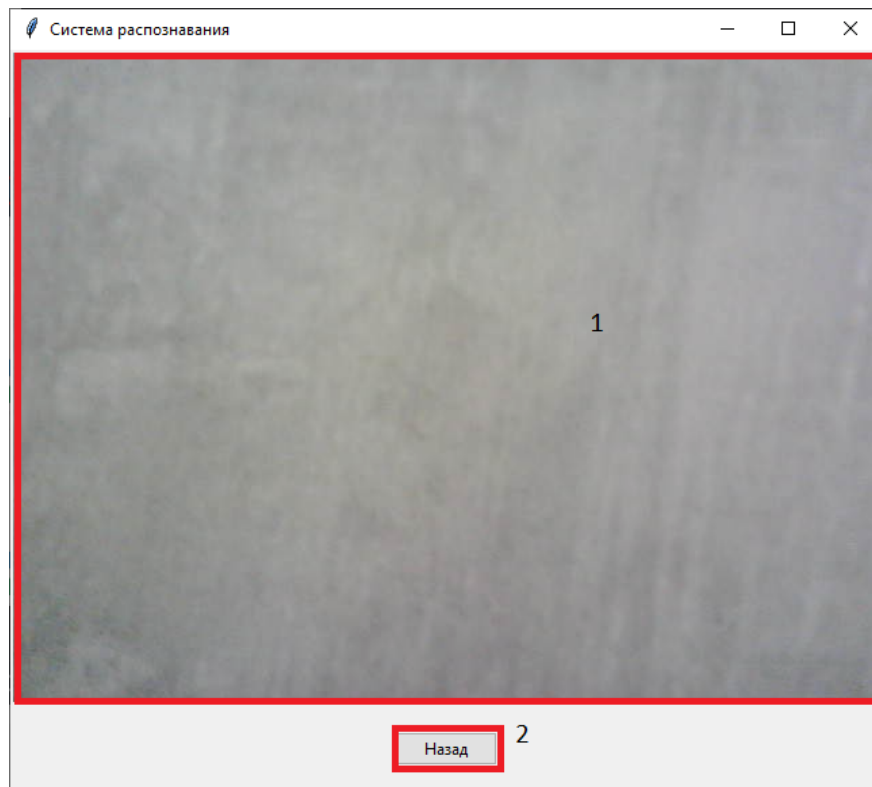


Рисунок 3.7 – Макет окна "Распознавание"

На рисунке 3.8 представлен макет интерфейса окна "Добавление пользователя". Макет содержит следующие элементы:

1. Поле для отображения текста с указаниями.
2. Поля для ввода имени пользователя.
3. Кнопка для запуска сканирования лица.
4. Кнопка для запуска сканирования отпечатка пальца.
5. Кнопка для создания карты доступа.
6. Кнопка выхода из программы.

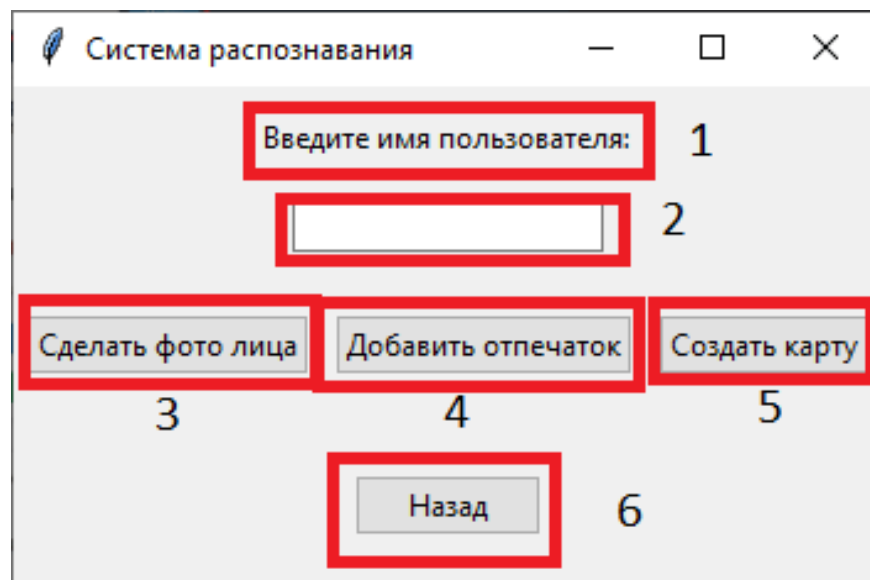


Рисунок 3.8 – Макет окна "Добавление пользователя"

4 Рабочий проект

4.1 Модули программной системы

4.1.1 Модуль `programm.py`

Модуль предоставляет основной графический интерфейс системы распознавания и управляет всеми процессами работы программы.

Класс - App.

Описание класса App: Класс является центральным компонентом системы, управляющим всеми функциями распознавания. Базовый класс – `tk.Tk`, стандартный класс библиотеки Tkinter. Интерфейсы – главное меню с тремя режимами работы, виджеты для добавления пользователя и распознавания, система обработки видео с камеры.

Внутренние поля класса представлены в таблице 4.1.

Таблица 4.1 – Внутренние поля класса App

Внутреннее поле	Тип поля	Описание поля
root	tk.Tk	Главное окно Tkinter
database	str	Путь к директории с файлами
fprints	str	Путь к директории отпечатков пальцев
embeddings_path	str	Путь к директории векторных признаков
cards_path	str	Путь к директории карт
faces_path	str	Путь к директории лиц
embeddings	dict	Словарь векторных признаков пользователей
device	torch.device	Устройство для вычислений
detector	MTCNN	Детектор лиц
inception	InceptionResnetV1	Модель векторных признаков лиц
fingerprints	Fingerprints	Объект для сравнения отпечатков
running	bool	Флаг, указывающий, работает ли приложение

Продолжение таблицы 4.1

Внутреннее поле	Тип поля	Описание поля
current_mode	str или None	Текущий режим приложения
current_user	str или None	Текущий распознанный пользователь
video_label	tk.Label или None	Метка для отображения видеопотока
cap	cv2.VideoCapture или None	Объект захвата видео
start_time	float или None	Метка времени для обработки кадров
user_name_entry	ttk.Entry или None	Поле ввода имени пользователя

Методы класса представлены в таблице 4.2.

Таблица 4.2 – Методы класса App

Название метода	Параметры метода	Возвращаемое значение	Описание метода
__init__	root, device, detector, inception, embeddings	Отсутствует	Инициализирует главное окно приложения
show_main_menu	Отсутствуют	Отсутствует	Отображает главное меню
start_add_user_mode	Отсутствуют	Отсутствует	Переключает в режим добавления пользователя
start_recognition_mode	Отсутствуют	Отсутствует	Переключает в режим распознавания
capture_user_face	Отсутствуют	Отсутствует	Захватывает изображения лица пользователя

Продолжение таблицы 4.2

Название метода	Параметры метода	Возвращаемое значение	Описание метода
add_fingerprint	Отсутствуют	Отсутствует	Добавляет отпечаток пальца
create_user_card	Отсутствуют	Отсутствует	Создает QR-код карты пользователя
show_qr_code	image_path: str — путь к изображению QR-кода	Отсутствует	Отображает QR-код в отдельном окне
card_reader	Отсутствуют	Отсутствует	Обрабатывает видеопоток для чтения карт
read_card	frame: np.ndarray — кадр видео	tuple[bool, str] — статус выполнения и декодированные данные	Распознает QR-код на кадре
process_detected_card	card: str — декодированные данные QR-кода	Отсутствует	Обрабатывает обнаруженную карту
verify_user	Отсутствуют	Отсутствует	Проверяет пользователя по лицу и отпечатку
clear_window	Отсутствуют	Отсутствует	Очищает окно приложения
on_close	Отсутствуют	Отсутствует	Завершает работу приложения

Класс - LoadingScreen.

Описание класса LoadingScreen: Отображает экран загрузки во время инициализации моделей. Базовый класс – tk.Tk, стандартный класс библиотеки Tkinter.

Внутренние поля класса представлены в таблице 4.3.

Таблица 4.3 – Внутренние поля класса LoadingScreen

Внутреннее поле	Тип поля	Описание поля
root	tk.Tk	Главное окно Tkinter
top	tk.Toplevel	Окно экрана загрузки
label	tk.Label	Метка с сообщением о загрузке

Методы класса представлены в таблице 4.4.

Таблица 4.4 – Методы класса LoadingScreen

Название метода	Параметры метода	Возвращаемое значение	Описание метода
destroy	Отсутствуют	Отсутствует	Закрывает экран загрузки

4.1.2 Модуль save_face.py

Модуль содержит функцию для захвата изображений лиц.

Методы модуля представлены в таблице 4.5.

Таблица 4.5 – Методы модуля save_face.py

Название метода	Параметры метода	Возвращаемое значение	Описание метода
capture_faces	username: str — имя пользователя для организации сохранённых изображений лица	tuple(bool, int) — статус выполнения и количество захваченных изображений	Захватывает и сохраняет изображения лица

4.1.3 Модуль `embedding_create.py`

Модуль содержит функции для работы с векторными представлениями лиц.

Класс – `FaceEmbedder`.

Описание класса `FaceEmbedder`: Генерирует векторные признаки лиц с использованием предобученных моделей MTCNN (детекция) и InceptionResnetV1 (кодирование). Базовый класс – отсутствует. Интерфейсы – отсутствуют.

Внутренние поля класса представлены в таблице 4.6.

Таблица 4.6 – Внутренние поля класса `FaceEmbedder`

Внутреннее поле	Тип поля	Описание поля
<code>device</code>	<code>torch.device</code>	Устройство для выполнения вычислений (CPU/GPU)
<code>mtcnn</code>	<code>facenet_pytorch.MTCNN</code>	Модель для обнаружения лиц
<code>inception</code>	<code>facenet_pytorch.InceptionResnetV1</code>	Модель извлечения признаков лица

Методы класса представлены в таблице 4.7.

Таблица 4.7 – Методы класса `FaceEmbedder`

Название метода	Параметры метода	Возвращаемое значение	Описание метода
<code>__init__</code>	Отсутствуют	Отсутствуют	Инициализирует модели для извлечения признаков лица
<code>get_embedding</code>	<code>img_path: str</code> — путь к изображению лица	<code>np.ndarray</code> — признаки лица, <code>None</code> — если обнаружение лица не удалось	Извлекает признаки лица из изображения

Методы модуля представлены в таблице 4.8.

Таблица 4.8 – Методы модуля embedding_create.py

Название метода	Параметры метода	Возвращаемое значение	Описание метода
update_user_embedding	username: str - имя пользователя	bool – статус выполнения	Вычисляет для указанного пользователя усредненное векторное представление лица по всем его изображениям и сохраняет в .пру-файл.
get_embeddings	Отсутствуют	dict[str, np.ndarray] — словарь, сопоставляющий имена пользователей их признакам лица.	Загружает все сохраненные векторные представления

4.1.4 Модуль func.py

Модуль содержит вспомогательные функции для обнаружения лиц, генерации векторных представлений и распознавания.

Методы модуля представлены в таблице 4.9.

Таблица 4.9 – Методы модуля func.py

Название метода	Параметры метода	Возвращаемое значение	Описание метода
load_models	Отсутствуют	tuple(torch.device, InceptionResnet, MTCNN) — устройство, модель признаков лица и детектор	Загружает модели MTCNN и InceptionResnetV1 для обнаружения лиц и генерации эмбеддингов

Продолжение таблицы 4.9

Название метода	Параметры метода	Возвращаемое значение	Описание метода
camera_connect	index: int — индекс камеры	cv2.VideoCapture — объект камеры	Пытается подключиться к камере по указанному индексу
detect_faces	img: np.ndarray — входное изображение, embeddings: dict — словарь признаков пользователей, detector: MTCNN — детектор лиц, device: torch.device — устройство для вычислений, inception: InceptionResnetV1 — модель признаков	list[dict] — список обнаруженных лиц с их координатами, уверенностью, именем и оценкой схожести	Обнаруживает лица на изображении и распознаёт их
load_embeddings	path: str — путь к директории с файлами признаков	dict[str, np.ndarray] — словарь, сопо- ставляющий имена пользователей признакам	Загружает все файлы признаков (.npy) из указанной директории

Продолжение таблицы 4.9

Название метода	Параметры метода	Возвращаемое значение	Описание метода
get_embedding	face_img: np.ndarray — изображение лица, device: torch.device — устройство для вычислений, inception: InceptionResnetV1 — модель признаков	np.ndarray или None — признаки лица или None при некорректном входе	Генерирует векторное представление для указанного изображения лица
recognize_face	face: np.ndarray — изображение лица, embeddings: dict — словарь признаков пользователей, device: torch.device — устройство для вычислений, inception: InceptionResnetV1 — модель признаков	tuple(str, float) — имя пользователя и оценка схожести	Распознаёт лицо, сравнивая его признаки с сохранёнными

Продолжение таблицы 4.9

Название метода	Параметры метода	Возвращаемое значение	Описание метода
cosine_similarity	vec: np.ndarray — тестовый вектор признаков, ref_vecs: np.ndarray — опорный вектор признаков	np.ndarray или float — оценка сходства	Вычисляет косинусное сходство между тестовым и опорным вектором

4.1.5 Модуль fp.py

Модуль предназначен для сравнения отпечатков пальцев с использованием сиамской сети.

Класс – Fingerprints.

Описание класса Fingerprints: Сравнивает изображения отпечатков пальцев с использованием предобученной сиамской сети. Базовый класс – отсутствует. Интерфейсы – отсутствуют.

Внутренние поля класса представлены в таблице 4.10.

Таблица 4.10 – Внутренние поля класса Fingerprints

Внутреннее поле	Тип поля	Описание поля
transform	torchvision.transforms.Compose	Трансформации для предобработки изображений
device	torch.device	Устройство для вычислений
model	SiameseNetwork	Предобученная модель сиамской сети
model_path	str	Путь к сохранённым весам модели

Методы класса представлены в таблице 4.11.

Таблица 4.11 – Методы класса Fingerprints

Название метода	Параметры метода	Возвращаемое значение	Описание метода
<code>__init__</code>	Отсутствуют	Отсутствует	Инициализирует модель для сравнения отпечатков
<code>compare_images</code> <code>_cosine</code>	<code>img_path1: str</code> — путь к первому изображению отпечатка, <code>img_path2: str</code> — путь ко второму изображению отпечатка	<code>float</code> — оценка косинусного сходства	Сравнивает два отпечатка по косинусной близости

4.1.6 Модуль `model.py`

Модуль определяет модели нейронных сетей для распознавания отпечатков пальцев – `FingerprintEncoder` и `SiameseNetwork`.

Класс – `FingerprintEncoder`.

Описание класса `FingerprintEncoder`: Кодировать изображения отпечатков пальцев в векторы признаков с использованием модели `ResNet18`. Базовый класс – `torch.nn.Module`. Интерфейсы – отсутствуют.

Внутренние поля класса представлены в таблице 4.12.

Таблица 4.12 – Внутренние поля класса `FingerprintEncoder`

Внутреннее поле	Тип поля	Описание поля
<code>encoder</code>	<code>torchvision.models.resnet.ResNet</code>	Предобученная модель <code>ResNet18</code> с заменённым полносвязным слоем на слой идентичности

Методы класса представлены в таблице 4.15.

Таблица 4.13 – Методы класса FingerprintEncoder

Название метода	Параметры метода	Возвращаемое значение	Описание метода
forward	x: torch.Tensor — входной тензор изображения	torch.Tensor — вектор признаков	Обрабатывает входное изображение через кодировщик ResNet18 для получения вектора признаков

Класс – SiameseNetwork.

Описание класса SiameseNetwork: Реализует сиамскую сеть для сравнения пар изображений отпечатков пальцев. Базовый класс – torch.nn.Module. Интерфейсы – отсутствуют. Константы – отсутствуют.

Внутренние поля класса представлены в таблице 4.14.

Таблица 4.14 – Внутренние поля класса SiameseNetwork

Внутреннее поле	Тип поля	Описание поля
encoder	FingerprintEncoder	Экземпляр класса FingerprintEncoder для извлечения признаков
fc	torch.nn.Sequential	Полносвязные слои для обработки разности векторов признаков и вывода оценки схожести

Методы класса представлены в таблице 4.15.

Таблица 4.15 – Методы класса SiameseNetwork

Название метода	Параметры метода	Возвращаемое значение	Описание метода
forward	img1: torch.Tensor — тензор первого изображения отпечатка, img2: torch.Tensor — тензор второго изображения отпечатка	torch.Tensor — оценка схожести от 0 до 1	Извлекает признаки из обоих изображений, вычисляет их абсолютную разность и пропускает через полносвязные слои для получения оценки схожести
forward_once	x: torch.Tensor — тензор одного изображения отпечатка	torch.Tensor — вектор признаков	Извлекает признаки из одного изображения с помощью кодировщика

4.1.7 Модуль dataset.py

Модуль содержит функции для загрузки и подготовки пар изображений отпечатков пальцев для обучения сиамской сети.

Класс – FingerprintPairsDataset.

Описание класса FingerprintPairsDataset: Создаёт пары изображений отпечатков пальцев (положительные и отрицательные пары) для обучения. Базовый класс – torch.utils.data.Dataset. Интерфейсы – отсутствуют.

Внутренние поля класса представлены в таблице 4.16.

Таблица 4.16 – Внутренние поля класса FingerprintPairsDataset

Внутреннее поле	Тип поля	Описание поля
mine_images	list[str]	Список путей к файлам изображений отпечатков пользователя

Продолжение таблицы 4.16

Внутреннее поле	Тип поля	Описание поля
others_images	list[str]	Список путей к файлам изображений отпечатков других пользователей
transform	torchvision.transforms.Compose	Трансформации для предобработки изображений
positive_pairs	list[tuple[str, str]]	Список пар изображений отпечатков одного пользователя
negative_pairs	list[tuple[str, str]]	Список пар, содержащих изображение пользователя и изображение другого человека
pairs	list[tuple[str, str]]	Объединённый список положительных и отрицательных пар
labels	list[int]	Метки (1 для положительных пар, 0 для отрицательных)

Методы класса представлены в таблице 4.17.

Таблица 4.17 – Методы класса FingerprintPairsDataset

Название метода	Параметры метода	Возвращаемое значение	Описание метода
__len__	Отсутствуют	int — количество пар	Возвращает общее количество пар изображений
__getitem__	idx: int — индекс пары	tuple(torch.Tensor, torch.Tensor, int) — два предобработанных тензора изображений и их метка	Загружает и преобразовывает пару изображений по указанному индексу, возвращает изображения и их метку

4.1.8 Модуль train.py

Модуль предназначен для обучения сиамской сети с целью распознавания отпечатков пальцев и предоставляет функцию для их сравнения.

Методы модуля представлены в таблице 4.18.

Таблица 4.18 – Методы модуля train.py

Название метода	Параметры метода	Возвращаемое значение	Описание метода
compare_images	model: SiameseNetwork — обученная модель сиамской сети, img_path1: str — путь к первому изображению отпечатка, img_path2: str — путь ко второму изображению отпечатка	float — оценка схожести	Вычисляет оценку схожести между двумя изображениями отпечатков с использованием сиамской сети

4.2 Модульное тестирование разработанного программного решения

Модульный тест для модуля model.py представлен в таблице 4.19.

Таблица 4.19 – Модульное тестирование модуля model.py

Описание теста	Входные данные	Ожидаемый результат
Проверка вывода FingerprintEncoder	Тензор размера (1, 3, 500, 500)	Тензор размера (1, 512)

Продолжение таблицы 4.19

Описание теста	Входные данные	Ожидаемый результат
Проверка работы SiameseNetwork	Два одинаковых тензора (1, 3, 500, 500)	Значение в диапазоне [0, 1]
Проверка метода forward_once	Тензор размера (1, 3, 500, 500)	Тензор размера (1, 512)

Модульный тест для модуля fr.py представлен в таблице 4.20.

Таблица 4.20 – Модульное тестирование модуля fr.py

Описание теста	Входные данные	Ожидаемый результат
Инициализация класса Fingerprints	Имитация модели SiameseNetwork и torch.load	Успешное создание экземпляра
Сравнение двух изображений отпечатков	Пути к двум черно-белым изображениям (500x500)	Значение сходства в диапазоне [-1, 1]

Модульный тест для модуля func.py представлен в таблице 4.21.

Таблица 4.21 – Модульное тестирование модуля func.py

Описание теста	Входные данные	Ожидаемый результат
Загрузка эмбеддингов из папки	Папка с двумя .пру файлами эмбеддингов	Словарь с ключами 'user1', 'user2' и значениями-массивами
Получение эмбеддинга из изображения	Набор имитаций: модель, трансформации, изображение numру	Эмбеддинг размером (512,)
Косинусное сходство между векторами	Два ортогональных вектора и два одинаковых	0 (ортогональные), 1 (одинаковые)

Модульный тест для модуля dataset.py представлен в таблице 4.22.

Таблица 4.22 – Модульное тестирование модуля dataset.py

Описание теста	Входные данные	Ожидаемый результат
Проверка длины датасета	Папки с 3 изображениями нужного человека и 3 другого	Длина равна 6
Проверка выдачи элемента датасета	Индекс 0	Два изображения размера (500, 500) и метка 0 или 1

Модульный тест для модуля embedding_create.py представлен в таблице 4.23.

Таблица 4.23 – Модульное тестирование модуля embedding_create.py

Описание теста	Входные данные	Ожидаемый результат
Получение эмбединга лица с имитациями моделей	Путь к изображению, имитация MTCNN, имитация модели InceptionResnetV1	Эмбединг размером (512,)
Обновление эмбединга пользователя	Имя пользователя	True

4.3 Системное тестирование разработанного программного решения

На рисунке 4.1 представлено главное окно интеллектуальной системы.

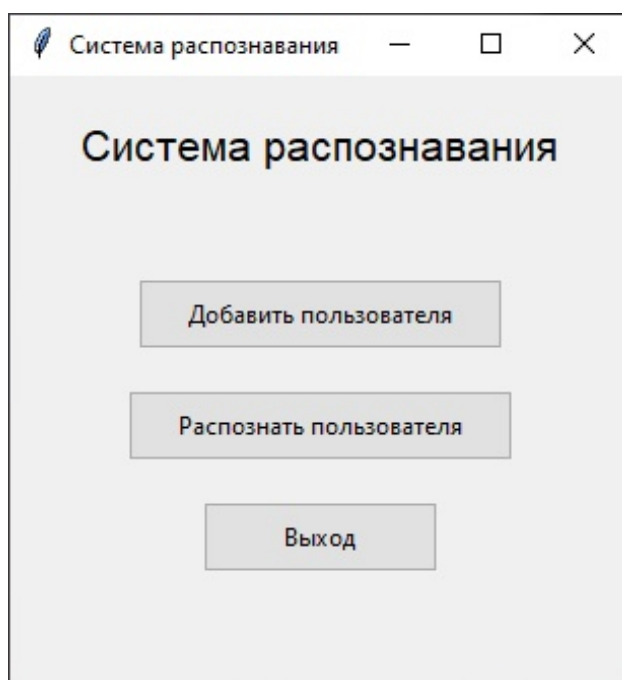


Рисунок 4.1 – Главное окно программы

На рисунке 4.2 представлено окно "Добавить пользователя".

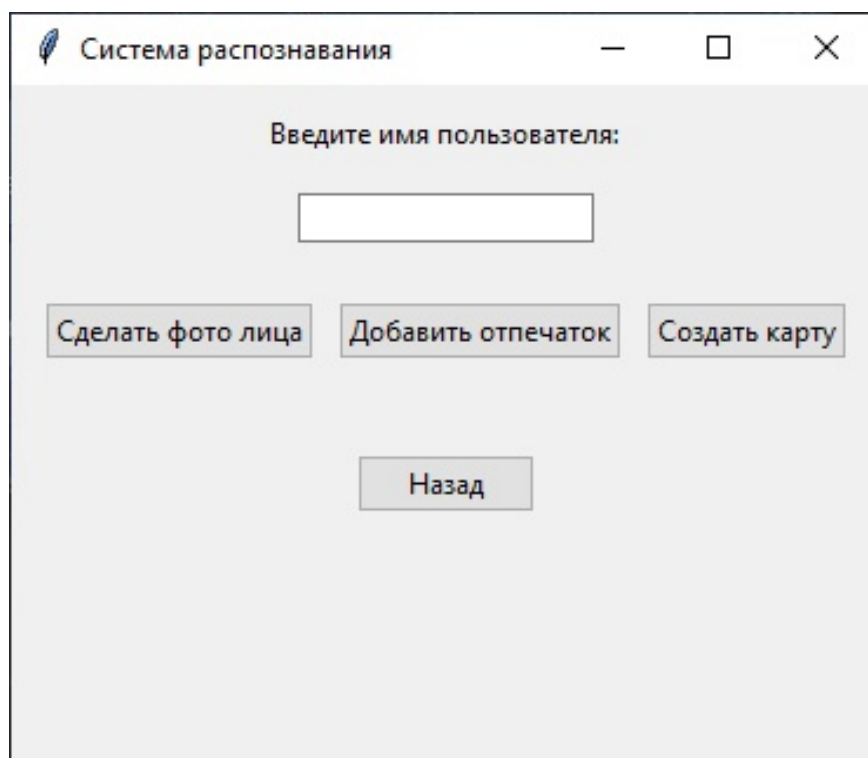


Рисунок 4.2 – Окно "Добавить пользователя"

На рисунке 4.3 представлено отображение ошибки при отсутствии введенного имени пользователя.

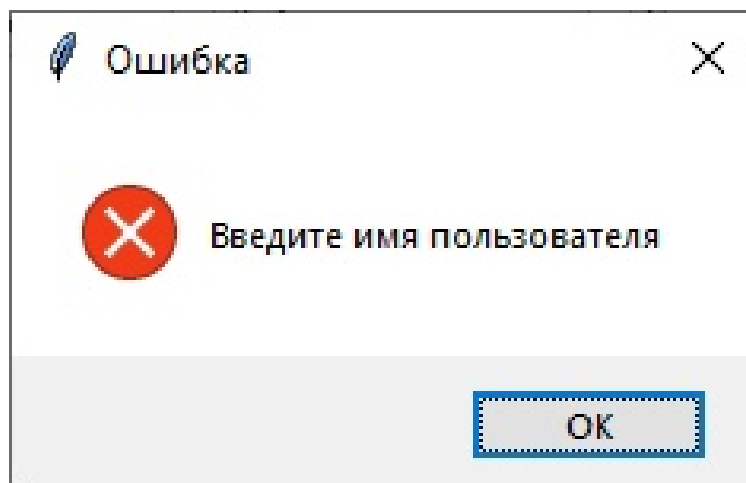


Рисунок 4.3 – Окно отображения ошибки ”Введите имя пользователя”

На рисунке 4.4 представлено отображение информации о сохранении признаков лица.

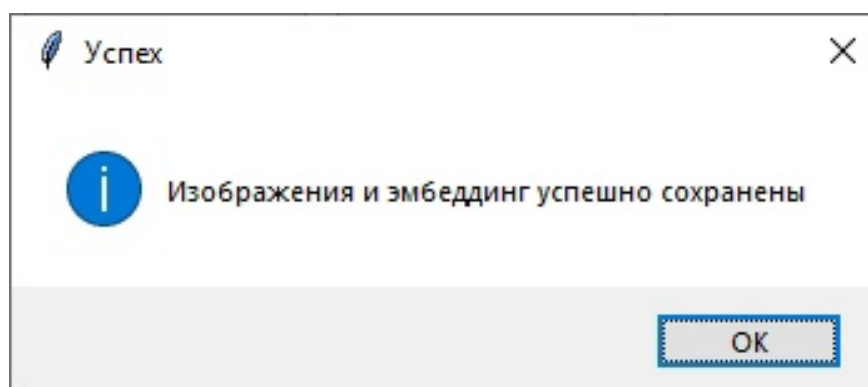


Рисунок 4.4 – Окно отображения информации о сохранении признаков лица

На рисунке 4.5 представлено диалоговое окно выбора изображения с отпечатком пальца.

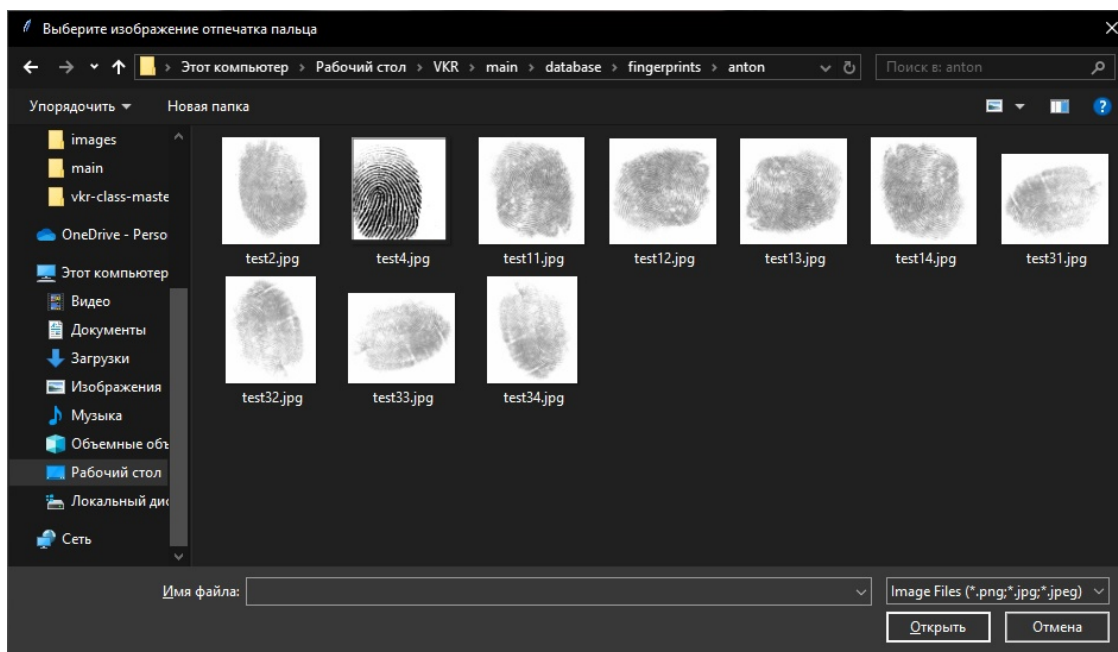


Рисунок 4.5 – Диалоговое окно выбора изображения с отпечатком пальца

На рисунке 4.6 представлено отображение информации о сохранении отпечатка пальца.

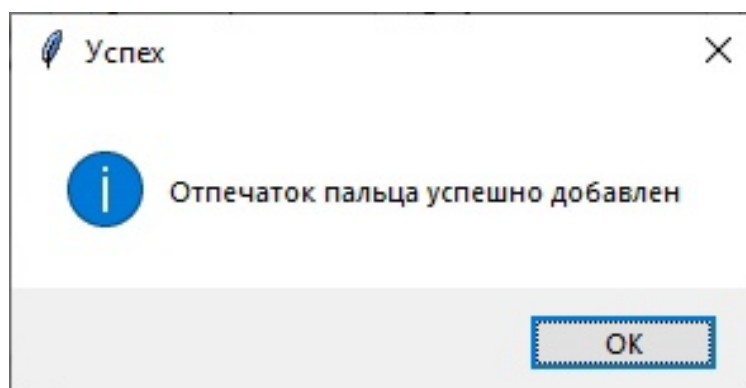


Рисунок 4.6 – Окно отображения информации о сохранении отпечатка пальца

На рисунке 4.7 представлено отображение информации о создании карты доступа.

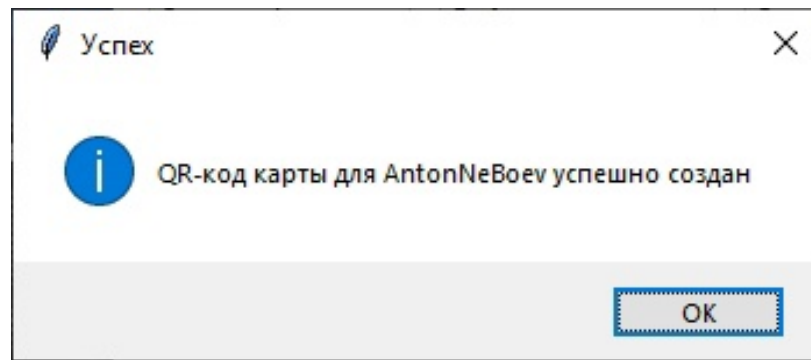


Рисунок 4.7 – Окно отображения информации о создании карты доступа

На рисунке 4.8 представлено созданное для пользователя изображение карты доступа.



Рисунок 4.8 – Окно отображения созданной карты

На рисунке 4.9 представлено окно "Распознать пользователя".

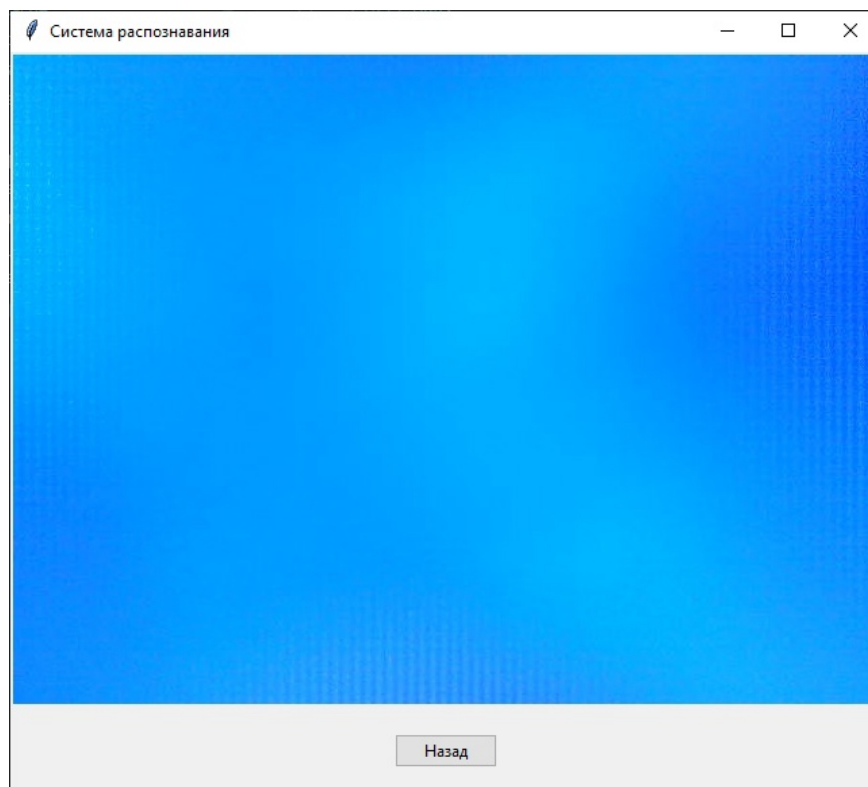


Рисунок 4.9 – Окно "Распознать пользователя"

На рисунке 4.10 представлено окно отображения ошибки при попытке использовать неподходящую карту.

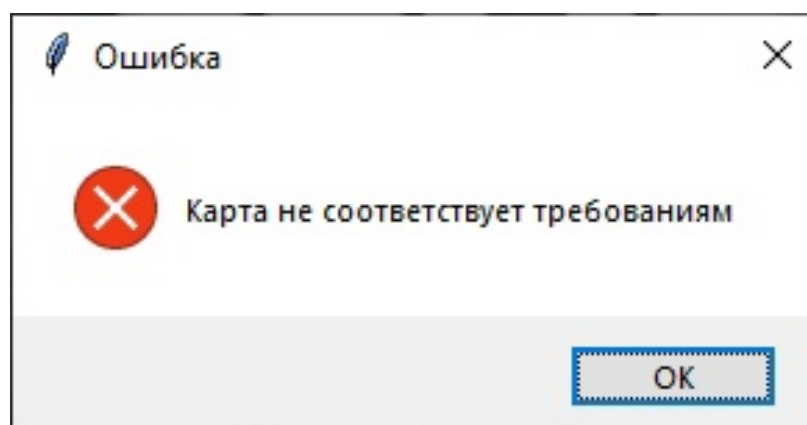


Рисунок 4.10 – Окно отображения ошибки "Карта не соответствует требованиям"

На рисунке 4.11 представлено окно отображения ошибки при попытке сканирования изображения лица, не принадлежащего владельцу карты.

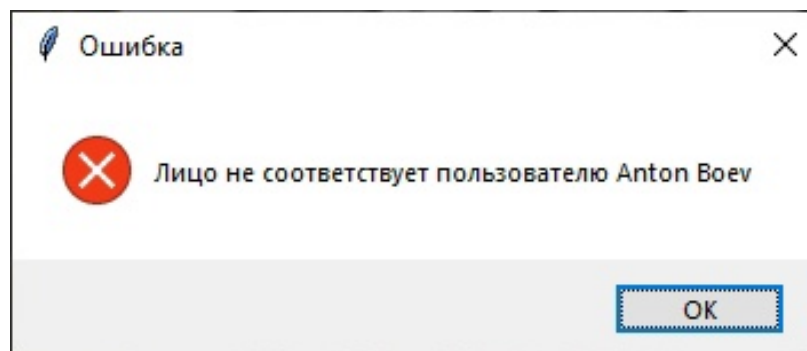


Рисунок 4.11 – Окно отображения ошибки ”Лицо не соответствует пользователю”

На рисунке 4.12 представлено окно отображения ошибки при использовании отпечатков, не принадлежащих пользователю.

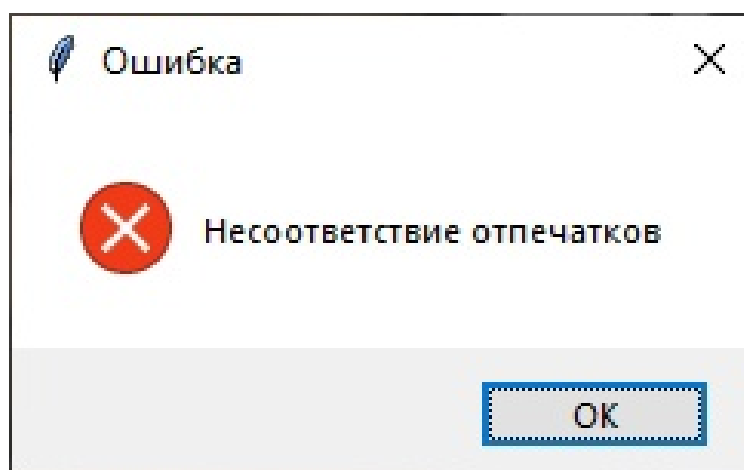


Рисунок 4.12 – Окно отображения ошибки ”Несоответствие отпечатков”

4.4 Сборка программной системы

Программные компоненты представляют собой файлы исходных кодов программной системы.

Для сборки и компиляции программной системы использовалась библиотека Pyinstaller, позволяющая упаковать все необходимые файлы в один исполняемый файл формата .exe. Данный файл может быть запущен без предварительной установки.

Интерпретация исходных кодов на языке Python выполняется встроенным в исполняемый файл интерпретатором языка и не требует отдельной установки интерпретатора и библиотек на целевую систему.

Все программные компоненты собраны в один исполняемый файл, готовый к запуску в среде Windows.

ЗАКЛЮЧЕНИЕ

Прогресс в развитии нейронных сетей и методов машинного обучения предоставил новые возможности для создания систем биометрической идентификации. Современные технологии позволяют эффективно обрабатывать биометрические данные, такие как изображения лиц и отпечатков пальцев, обеспечивая высокую точность и надёжность аутентификации.

В условиях роста потребности в безопасных системах идентификации разработка интеллектуальных решений для распознавания пользователей является актуальной задачей. Применение таких систем позволяет повысить уровень безопасности, упростить процесс аутентификации и снизить затраты на реализацию.

Для решения задачи биометрической идентификации была разработана интеллектуальная система, основанная на использовании сверточной нейронной сети ResNet18 в составе сиамской архитектуры для распознавания отпечатков пальцев и модели InceptionResnetV1 для обработки лиц. Система реализована в виде настольного приложения с графическим интерфейсом.

Основные результаты работы:

1. Проведен анализ предметной области и существующих решений. Выбраны методы распознавания.
2. Разработана концептуальная модель интеллектуальной системы, определены основные требования к системе.
3. Спроектированы модуль захвата, обработки и распознавания лиц на изображении, модуль распознавания отпечатков пальцев и механизм обработки карт доступа.
4. Реализована интеллектуальная система распознавания, проведено модульное и системное тестирование разработанной системы.

Все требования, объявленные в техническом задании, были полностью реализованы. Все задачи, поставленные в начале разработки проекта, были решены.

Готовый рабочий проект представлен в виде настольного приложения с графическим интерфейсом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Большаков, И. А. Системы распознавания лиц – принцип работы и сферы применения / И. А. Большаков, С. А. Микаева. – Текст : непосредственный // Науносфера. – 2022. – № 9-2. – С. 95–98.
2. Ветров, С. В. История развития систем распознавания лиц / С. В. Ветров. – Текст : непосредственный // Наука и образование: актуальные исследования и разработки : материалы IV Всероссийской научно-практической конференции, Чита, 2021 / отв. ред. А. В. Лесков ; Забайкальский государственный университет. – Чита : ЗабГУ, 2021. – С. 23–29.
3. Байкенов, Б. С. Сравнительный анализ методов распознавания объектов / Б. С. Байкенов, Р. С. Тынчеров, А. Р. Фазылова. – Текст : непосредственный // Вестник Казахской академии транспорта и коммуникаций им. М. Тынышпаева. – 2019. – № 1 (108). – С. 185–191.
4. Романюта, Д. Ю. Исследование алгоритма Виолы-Джонса для разработки системы распознавания лиц с помощью нейронных сетей / Д. Ю. Романюта, А. В. Коваленко, М. В. Шарпан. – Текст : непосредственный // Инженерный вестник Дона. – 2024. – № 1 (109). – С. 739–751.
5. Смирнов, Н. А. Обзор существующих методов обнаружения объектов / Н. А. Смирнов ; Казанский (Приволжский) федеральный университет. – Текст : непосредственный // Вестник науки. – 2024. – Т. 3, № 9 (78). – С. 386–389.
6. Нестерова, Т. Г. Современные детекторы лиц: обзор / Т. Г. Нестерова ; ФГАОУ ВО «Новосибирский национальный исследовательский государственный университет». – Текст : непосредственный // Наука и технологии – 2023 : сб. статей Международной научно-практической конференции, Петрозаводск, 2023. – Петрозаводск : Международный центр научного партнерства «Новая Наука» (ИП Ивановская И.И.), 2023. – ISBN 978-5-00174-958-5. – С. 161–174.

7. Абдуллаев, А. И. Распознавание лиц по изображению лица / А. И. Абдуллаев. – Текст : непосредственный // Мировая наука. – 2021. – № 4 (49). – С. 44–47.

8. Кандидата наук Ермошина задержали как вора из-за ошибки распознавания лиц [Электронный ресурс] // Московский комсомолец. – 2021. – 19 октября. – URL: <https://www.mk.ru/incident/2021/10/19/kandidata-nauk-ermoshina-zaderzhali-kak-vora-izza-oshibki-raspoznavaniya-lic.html> (дата обращения: 03.05.2025). – Текст : электронный.

9. Вигерс, К. Разработка требований к программному обеспечению : [практические приёмы сбора требований и управления ими при разработке программных продуктов : пер. с англ.] / К. Вигерс, Д. Битти. – 3-е изд., доп. – Санкт-Петербург : BHV, 2020. – 736 с. – ISBN 978-5-9775-3348-5. – Текст : непосредственный.

10. Рашка, С. Python и машинное обучение / С. Рашка. – Санкт-Петербург : БХВ-Петербург, 2019. – 416 с. – ISBN 978-5-97060-409-0. – Текст : непосредственный.

11. Хайкин, С. Нейронные сети: полный курс / Саймон Хайкин – 2-е изд., испр. – М. : Вильямс, 2019. – 1104 с. – ISBN 978-5-907144-22-4. – Текст : непосредственный.

12. Фаулер, М. UML. Основы / М. Фаулер. – 3-е изд. – Санкт-Петербург : Символ-Плюс, 2015. – 192 с. – ISBN 978-5-93286-060-1. – Текст : непосредственный.

13. Лутц, М. Изучаем Python. Том 1 : учебное пособие / М. Лутц. – 5-е изд. – М. : Вильямс, 2020. – 832 с. – ISBN 978-5-907144-52-1. – Текст : непосредственный.

14. Васильев, А. Н. Программирование на Python в примерах и задачах / А. Н. Васильев. – Москва : Бомбора, 2021. – 384 с. – ISBN 978-5-04-103199-2. – Текст : непосредственный.

15. Бендер, Д. Python для анализа данных. – М. : ДМК Пресс, 2015. – 482 с. – ISBN 978-5-97060-315-4. – Текст : непосредственный.

16. Лутц, М. Изучаем Python. Том 2 / М. Лутц. – 5-е изд. – М. : Вильямс, 2020. – 720 с. – ISBN 978-5-907144-53-8. – Текст : непосредственный.
17. Пойнтер, Я. Программируем с PyTorch: создание приложений глубокого обучения / Я. Пойнтер. – Санкт-Петербург : Питер, 2020. – 256 с. – ISBN 978-5-4461-1677-5. – Текст : непосредственный.
18. Шолле, Ф. Глубокое обучение на Python / пер. с англ. – М. : ДМК Пресс, 2018. – 384 с. – ISBN 978-5-4461-0770-4. – Текст : непосредственный.
19. Киба, А. С. Использование сиамских нейронных сетей в задаче распознавания лиц / А. С. Киба. – Текст : непосредственный // Решетнёвские чтения : материалы XXIII Междунар. науч.-практ. конф., посвящ. памяти ген. конструктора ракетно-космич. систем акад. М. Ф. Решетнева, 11–15 нояб. 2019 г., г. Красноярск : в 2 ч. Ч. 2 / под общ. ред. Ю. Ю. Логинова ; СибГУ им. акад. М. Ф. Решетнева. – Красноярск : СибГУ им. акад. М. Ф. Решетнева, 2019. – ISBN 978-5-86433-790-5. – С. 348–349.
20. Иванова, Г. С. Проектирование программного обеспечения : учебное пособие / Г. С. Иванова, Т. Н. Ничушкина. – Москва : МГТУ им. Н. Э. Баумана, 2003. – 102,[1] с. – ISBN 5-7038-2285-8. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.11.

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи

Цель данной выпускной квалификационной работы – разработка интеллектуальной системы, объединяющей методы компьютерного зрения и машинного обучения для распознавания лиц и отпечатков пальцев, интегрированной с картами доступа.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и существующих решений.
2. Разработать концептуальную модель системы и выбрать методы распознавания.
3. Спроектировать архитектуру программной системы распознавания на основе изображений лиц и отпечатков пальцев.
4. Реализовать приложение, используя графический интерфейс.

				ВКРБ 2106138.09.03.04.25.006		
				Дата	Место	Подпись
Имя	Фамилия	Отчество	Подпись	Цели и задачи		
Имя	Фамилия	Отчество	Подпись	Лист 2	Лист 11	
				Выпускная квалификационная работа		
				03/19 ПО-116		

Рисунок А.2 – Цель и задачи работы

80

P-Net

Conv 3*3
MP 2*2

Conv 3*3

Conv 3*3

1*1*2
1*1*4
1*1*1

Форма входа 12*12*3

5*5*10

3*3*16

1*1*32

R-Net

Conv 3*3
MP 3*3

Conv 3*3
MP 3*3

Conv 2*2

FC

2
4
10

Форма входа 24*24*3

11*11*28

4*4*48

3*3*64

128

O-Net

Conv 3*3
MP 3*3

Conv 3*3
MP 3*3

Conv 3*3
MP 2*2

Conv 2*2

FC

2
4
10

Форма входа 48*48*3

23*23*32

10*10*64

4*4*64

3*3*128

256

Рисунок А.5 – Архитектура МТСNN

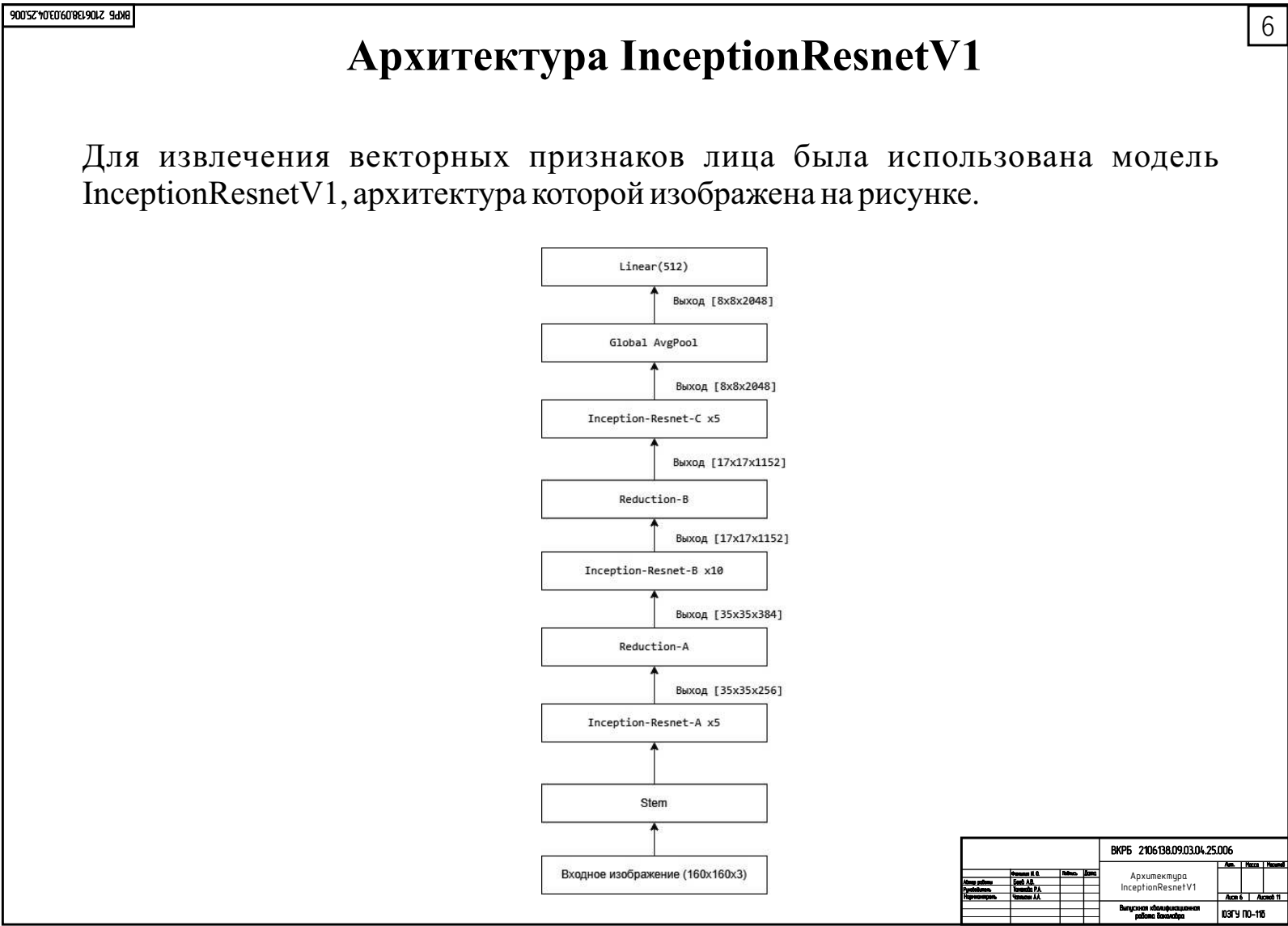


Рисунок А.6 – Архитектура InceptionResnetV1

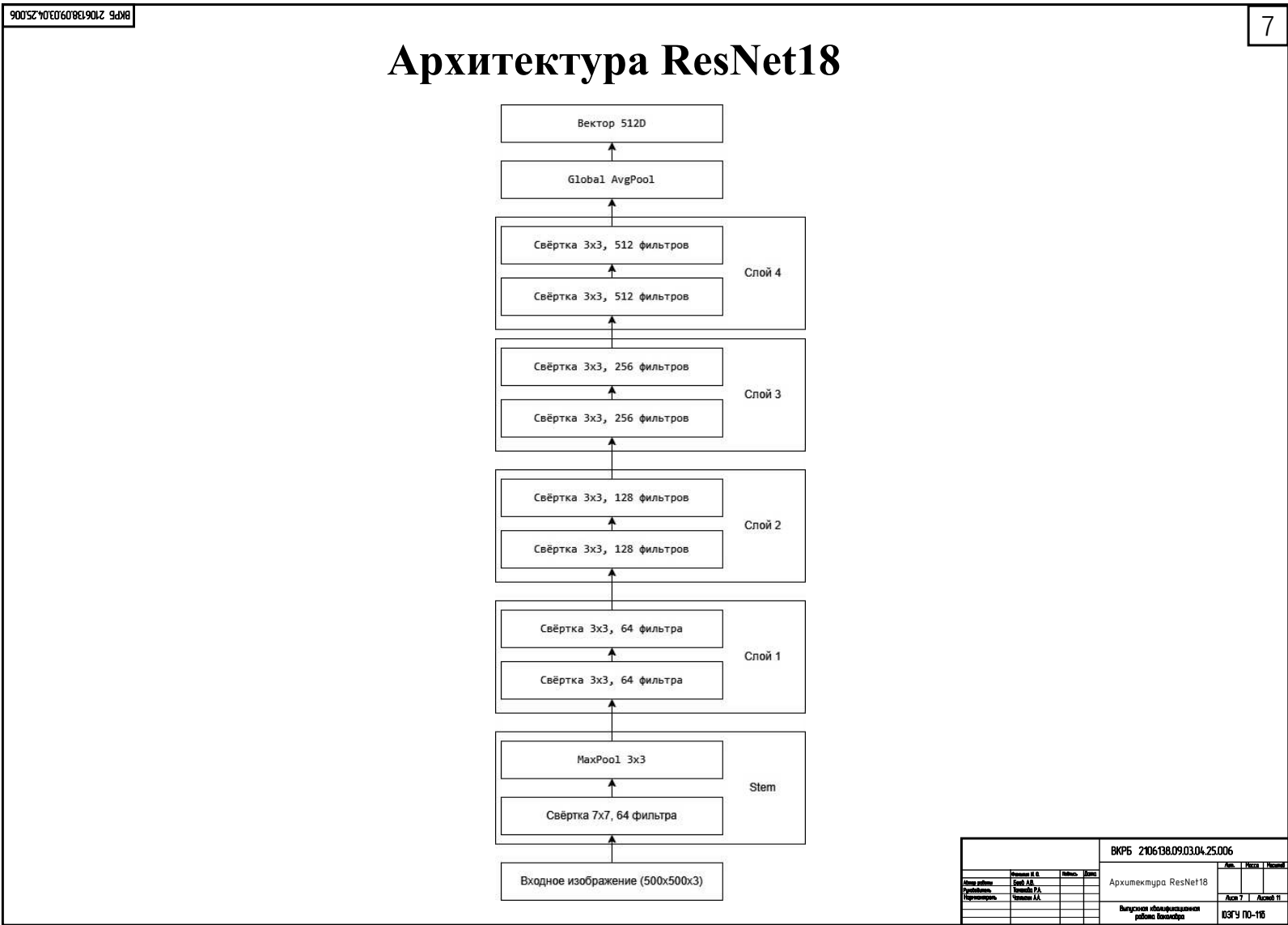


Рисунок А.7 – Архитектура ResNet18

Метод получения изображения отпечатков пальцев

Для получения изображения отпечатков пальцев в домашних условиях можно использовать графитовый порошок от карандаша и скотч. Палец необходимо прокатить по заштрихованной поверхности, перенести отпечаток на чистую бумагу при помощи клейкой ленты, после чего оцифровать при помощи сканера.



ВКРБ 2106188.09.03.04.25.006			
Метод получения изображения отпечатков пальцев		Акт	Исход
Выполнен специалистом		Акт 1	Акт 2
работы: Кошарова		08/11/16	
Исполнитель	Секрет	Метод	Исход
Исполнитель	Секрет	Метод	Исход
Исполнитель	Секрет	Метод	Исход

Рисунок А.9 – Метод получения изображения отпечатков пальцев

ВКРБ 2106188.09.03.04.25.006

10

Главные окна программы

На слайде изображены главные окна разработанной системы распознавания.

Система распознавания

Добавить пользователя

Распознать пользователя

Выход

Система распознавания

Введите имя пользователя:

Сделать фото лица

Добавить отпечаток

Создать карту

Назад

				ВКРБ 2106188.09.03.04.25.006			
				Авт. Искл. Нормат.			
				Авт. 0 Авт. 1 Авт. 2			
				Выпускная квалификационная работа бакалавра 03/У ПО-16			

Рисунок А.10 – Главные окна программы

Заключение

Основные результаты работы:

1. Проведен анализ предметной области и существующих решений. Выбраны методы распознавания.
2. Разработана концептуальная модель интеллектуальной системы, определены основные требования к системе.
3. Спроектированы модуль захвата, обработки и распознавания лиц на изображении, модуль распознавания отпечатков пальцев и механизм обработки карт доступа.
4. Реализована интеллектуальная система распознавания, проведено модульное и системное тестирование разработанной системы.

Все задачи, поставленные в начале разработки проекта, были решены.

				ВКРБ 2106138.09.03.04.25.006			
				Заключение			
				Выпускная квалификационная работа выполнена			
				ОБЗУ ПО-116			

Рисунок А.11 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

programm.py

```
1 import os
2 import cv2
3 import time
4 import tkinter as tk
5 from tkinter import messagebox, filedialog, ttk
6 from PIL import Image, ImageTk
7 import qrcode
8 import threading
9 import func
10 import fp
11 from save_face import capture_faces
12 from embedding_create import get_embeddings
13
14 class App:
15     def __init__(self, root, device, detector, inception, embeddings):
16         self.root = root
17         self.root.title("Система распознавания")
18
19         base_dir = os.path.dirname(os.getcwd())
20         self.database = os.path.join(base_dir, "database")
21         self.fprints = os.path.join(self.database, "fingerprints")
22         self.embeddings_path = os.path.join(self.database, "embeddings")
23         self.cards_path = os.path.join(self.database, "cards")
24         self.faces_path = os.path.join(self.database, "faces")
25
26         self.embeddings = embeddings
27         self.device = device
28         self.detector = detector
29         self.inception = inception
30         self.fingerprints = fp.Fingerprints()
31
32         self.running = True
33         self.current_mode = None
34         self.current_user = None
35
36         self.show_main_menu()
37
38     def show_main_menu(self):
39         """Главное меню с выбором режима"""
40         self.clear_window()
41         self.current_mode = None
42
43         title = tk.Label(self.root, text="Система распознавания", font=(
44             "Arial", 16))
45         title.pack(pady=20)
46
47         btn_frame = tk.Frame(self.root)
48         btn_frame.pack(pady=20)
```

```

49         ttk.Button(btn_frame, text="Добавить пользователя",
50                     command=self.start_add_user_mode).pack(pady=10, ipadx=20,
51                                                             ipady=5)
52
53         ttk.Button(btn_frame, text="Распознать пользователя",
54                     command=self.start_recognition_mode).pack(pady=10, ipadx=20,
55                                                             ipady=5)
56
57         ttk.Button(btn_frame, text="Выход",
58                     command=self.on_close).pack(pady=10, ipadx=20, ipady=5)
59
60     def start_add_user_mode(self):
61         """Режим добавления пользователя"""
62         self.current_mode = 'add_user'
63         self.clear_window()
64
65         tk.Label(self.root, text="Введите имя пользователя:").pack(pady=10)
66         self.user_name_entry = ttk.Entry(self.root)
67         self.user_name_entry.pack(pady=5)
68
69         btn_frame = tk.Frame(self.root)
70         btn_frame.pack(pady=20)
71
72         ttk.Button(btn_frame, text="Сделать фото лица",
73                     command=self.capture_user_face).pack(side=tk.LEFT, padx=5)
74
75         ttk.Button(btn_frame, text="Добавить отпечаток",
76                     command=self.add_fingerprint).pack(side=tk.LEFT, padx=5)
77
78         ttk.Button(btn_frame, text="Создать карту",
79                     command=self.create_user_card).pack(side=tk.LEFT, padx=5)
80
81         ttk.Button(self.root, text="Назад",
82                     command=self.show_main_menu).pack(pady=20)
83
84     def start_recognition_mode(self):
85         """Режим распознавания пользователя"""
86         self.current_mode = 'recognize'
87         self.clear_window()
88
89         self.video_label = tk.Label(self.root)
90         self.video_label.pack()
91
92         ttk.Button(self.root, text="Назад",
93                     command=self.show_main_menu).pack(pady=20)
94
95         self.cap = func.camera_connect(0)
96         if not self.cap.isOpened():
97             messagebox.showerror("Ошибка", "Не удалось открыть камеру")
98             self.show_main_menu()
99             return
100
101         self.start_time = time.time()
102         self.card_reader()

```



```

101
102 def capture_user_face(self):
103     username = self.user_name_entry.get().strip()
104     if not username:
105         messagebox.showerror("Ошибка", "Введите имя пользователя")
106         return
107
108     # Захват изображений лица
109     success = capture_faces(username)
110
111     if success:
112         # Обновление эмбединга
113         from embedding_create import update_user_embedding
114         if update_user_embedding(username):
115             messagebox.showinfo("Успех", "Изображения и эмбединг успешно
116                                     сохранены")
117         else:
118             messagebox.showwarning("Предупреждение", "Изображения
119                                     сохранены, но не удалось обновить эмбединг")
120     else:
121         messagebox.showerror("Ошибка", "Не удалось сохранить изображения")
122
123
124 def add_fingerprint(self):
125     """Добавление отпечатка пальца"""
126     username = self.user_name_entry.get().strip()
127     if not username:
128         messagebox.showerror("Ошибка", "Введите имя пользователя")
129         return
130
131     user_fprints_path = os.path.join(self.fprints, username)
132     os.makedirs(user_fprints_path, exist_ok=True)
133
134     file_path = filedialog.askopenfilename(
135         title="Выберите изображение отпечатка пальца",
136         filetypes=[("Image Files", "*.png;*.jpg;*.jpeg")]
137     )
138
139     if file_path:
140         try:
141             dest_path = os.path.join(user_fprints_path, "f1.jpg")
142             img = Image.open(file_path)
143             img.save(dest_path)
144             messagebox.showinfo("Успех", "Отпечаток пальца успешно
145                                     добавлен")
146         except Exception as e:
147             messagebox.showerror("Ошибка", f"Не удалось сохранить
148                                     отпечаток: {str(e)}")
149
150
151 def create_user_card(self):
152     """Создание QR-кода карты пользователя"""
153     username = self.user_name_entry.get().strip()
154     if not username:
155         messagebox.showerror("Ошибка", "Введите имя пользователя")

```

```

150         return
151
152     # Создаем папку для карт пользователя
153     user_cards_path = os.path.join(self.cards_path, username)
154     os.makedirs(user_cards_path, exist_ok=True)
155
156     # Генерируем QR-код
157     try:
158         # Создаем объект QR-кода
159         qr = qrcode.QRCode(
160             version=1,
161             error_correction=qrcode.constants.ERROR_CORRECT_L,
162             box_size=10,
163             border=4,
164         )
165
166         # Добавляем данные (используем имя пользователя как идентификатор)
167         qr.add_data(username)
168         qr.make(fit=True)
169
170         # Создаем изображение QR-кода
171         img = qr.make_image(fill_color="black", back_color="white")
172
173         # Сохраняем QR-код в файл
174         card_path = os.path.join(user_cards_path, f"{username}_card.png")
175         img.save(card_path)
176
177         # Показываем пользователю сгенерированный QR-код
178         self.show_qr_code(card_path)
179
180         messagebox.showinfo("Успех", f"QR-код карты для {username} успешно создан")
181         self.show_main_menu()
182
183     except Exception as e:
184         messagebox.showerror("Ошибка", f"Не удалось создать QR-код: {str(e)}")
185
186     def show_qr_code(self, image_path):
187         """Показывает QR-код в отдельном окне"""
188         qr_window = tk.Toplevel(self.root)
189         qr_window.title("Ваш QR-код")
190
191         # Загружаем изображение
192         img = Image.open(image_path)
193         img = ImageTk.PhotoImage(img)
194
195         # Создаем и размещаем элементы интерфейса
196         label = tk.Label(qr_window, image=img)
197         label.image = img # сохраняем ссылку на изображение
198         label.pack(padx=10, pady=10)
199
200         # Кнопка закрытия

```

```

201     close_btn = ttk.Button(qr_window, text="Заккрыть",
202                             command=qr_window.destroy)
203     close_btn.pack(pady=5)
204
205     def card_reader(self):
206         """Чтение карты и распознавание лица"""
207         def process_frame():
208             if not self.running or self.current_mode != 'recognize':
209                 return
210
211             ret, frame = self.cap.read()
212             if not ret:
213                 self.root.after(100, process_frame)
214                 return
215
216             frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
217             img = Image.fromarray(frame_rgb)
218             imgtk = ImageTk.PhotoImage(image=img)
219             self.video_label.imgtk = imgtk
220             self.video_label.configure(image=imgtk)
221
222             if time.time() - self.start_time > 1.0:
223                 self.start_time = time.time()
224                 detected, card = self.read_card(frame)
225                 if detected:
226                     self.process_detected_card(card)
227
228             self.root.after(30, process_frame)
229
230         process_frame()
231
232     def read_card(self, frame):
233         """Распознавание QR-кода"""
234         detector = cv2.QRCodeDetector()
235         data, _, _ = detector.detectAndDecode(frame)
236         return (True, data) if data else (False, None)
237
238     def process_detected_card(self, card):
239         """Обработка обнаруженной карты"""
240         card_file = os.path.join(self.cards_path, card, f"{card}_card.png")
241         if os.path.isfile(card_file):
242             self.current_user = card
243             messagebox.showinfo("Успех", f"Карта определена")
244             self.verify_user()
245         else:
246             messagebox.showerror("Ошибка", f"Карта не соответствует
247                                     требованиям")
248
249     def verify_user(self):
250         """Проверка пользователя по лицу и отпечатку"""
251         ret, frame = self.cap.read()
252         if not ret:
253             return

```

```

254     faces = func.detect_faces(frame, self.embeddings, self.detector, self
255                               .device, self.inception)
256     print(faces)
257     face_verified = any(face['name'] == self.current_user and face['score
258                          ']' > 0.8 for face in faces)
259
260     if face_verified:
261         fingerprint_path = os.path.join(self.fprints, self.current_user,
262                                         "f1.jpg")
263         if os.path.exists(fingerprint_path):
264             file_path = filedialog.askopenfilename(
265                 title="Выберите изображение отпечатка для проверки",
266                 filetypes=[("Image Files", "*.png;*.jpg;*.jpeg")]
267             )
268             if file_path:
269                 score = self.fingerprints.compare_images_cosine(file_path
270                                                                , fingerprint_path)
271                 if score > 0.8:
272                     messagebox.showinfo("Успех", f"Пользователь {self.
273                                              current_user} идентифицирован!")
274                 else:
275                     messagebox.showerror("Ошибка", "Несоответствие
276                                           отпечатков")
277             else:
278                 messagebox.showerror("Ошибка", "Отпечаток не найден в базе")
279         else:
280             messagebox.showerror("Ошибка", f"Лицо не соответствует
281                                       пользователю {self.current_user}")
282
283     self.current_user = None
284
285     def clear_window(self):
286         """Очистка окна"""
287         for widget in self.root.winfo_children():
288             widget.destroy()
289
290         if hasattr(self, 'cap') and self.cap.isOpened():
291             self.cap.release()
292
293     def on_close(self):
294         """Завершение работы"""
295         self.running = False
296         if hasattr(self, 'cap') and self.cap.isOpened():
297             self.cap.release()
298         self.root.destroy()
299
300     class LoadingScreen:
301         def __init__(self):
302             self.root = tk.Tk()
303             self.root.withdraw()
304
305             self.top = tk.Toplevel(self.root)
306             self.top.title("Загрузка...")

```

```

301     self.top.geometry("300x100")
302     self.top.resizable(False, False)
303     self.label = tk.Label(self.top, text="Загрузка моделей, подождите..."
304         , font=("Arial", 12))
305     self.label.pack(expand=True, padx=20, pady=30)
306     self.top.protocol("WM_DELETE_WINDOW", lambda: None)
307     self.top.grab_set()
308
309     def destroy(self):
310         self.root.destroy()
311
312 class LoadingScreen:
313     def __init__(self):
314         self.root = tk.Tk()
315         self.root.withdraw()
316
317         self.top = tk.Toplevel(self.root)
318         self.top.title("Загрузка...")
319         self.top.geometry("300x100")
320         self.top.resizable(False, False)
321         self.label = tk.Label(self.top, text="Загрузка моделей, подождите..."
322             , font=("Arial", 12))
323         self.label.pack(expand=True, padx=20, pady=30)
324         self.top.protocol("WM_DELETE_WINDOW", lambda: None)
325         self.top.grab_set()
326
327     def destroy(self):
328         self.top.destroy()
329         self.root.destroy()
330
331 if __name__ == "__main__":
332     loading = LoadingScreen()
333
334     app_data = {}
335
336     def background_load():
337         device, detector, inception = func.load_models()
338         embeddings_path = os.path.join(os.getcwd(), "database", "embeddings")
339         embeddings = func.load_embeddings(embeddings_path)
340         app_data["device"] = device
341         app_data["detector"] = detector
342         app_data["inception"] = inception
343         app_data["embeddings"] = embeddings
344         loading.root.after(0, start_main_app)
345
346     def start_main_app():
347         loading.destroy()
348         root = tk.Tk()
349         app = App(root,
350             app_data["device"],
351             app_data["detector"],
352             app_data["inception"],
353             app_data["embeddings"])
354         root.protocol("WM_DELETE_WINDOW", app.on_close)

```

```

353         root.mainloop()
354
355     threading.Thread(target=background_load, daemon=True).start()
356     loading.root.mainloop()

```

fp.py

```

1  import os
2  import torch.nn.functional as F
3  import torch
4  from PIL import Image
5
6  class Fingerprints:
7      def __init__(self):
8          from model import SiameseNetwork
9          import torchvision.transforms as transforms
10
11         self.transform = transforms.Compose([
12             transforms.Grayscale(num_output_channels=3),
13             transforms.Resize((500, 500)),
14             transforms.ToTensor()
15         ])
16
17         self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
18
19         self.model = SiameseNetwork().to(self.device)
20         self.model_path = os.path.join(os.getcwd(), "
                siamese_fingerprint_model.pth")
21         self.model.load_state_dict(torch.load(self.model_path, map_location=
                self.device))
22
23
24     def compare_images_cosine(self, img_path1, img_path2):
25         self.model.eval()
26         img1 = Image.open(img_path1).convert('L')
27         img2 = Image.open(img_path2).convert('L')
28
29         img1 = self.transform(img1).unsqueeze(0).to(self.device)
30         img2 = self.transform(img2).unsqueeze(0).to(self.device)
31
32         with torch.no_grad():
33             emb1 = self.model.forward_once(img1)
34             emb2 = self.model.forward_once(img2)
35             sim = F.cosine_similarity(emb1, emb2).item()
36
37         return sim

```

func.py

```

1  import os
2  import cv2
3  import numpy as np
4  import torch
5  from torchvision import transforms
6

```

```

7 def load_models():
8     from facenet_pytorch import MTCNN, InceptionResnetV1
9
10    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
11    detector = MTCNN(keep_all=True, device=device)
12    inception = InceptionResnetV1(pretrained='vggface2').to(device).eval()
13    return device, detector, inception
14
15 def camera_connect(index):
16     try:
17         cap = cv2.VideoCapture(index)
18         print("Камера найдена")
19     except Exception:
20         print("Камера не найдена")
21     return cap
22
23 def detect_faces(img, embeddings, detector, device, inception):
24     boxes, probs = detector.detect(img)
25     faces = []
26     if boxes is not None:
27         for box, prob in zip(boxes, probs):
28             x1, y1, x2, y2 = map(int, box)
29             face_img = img[y1:y2, x1:x2]
30             name, score = recognize_face(face_img, embeddings, device,
31                                         inception)
32             faces.append({
33                 'box': box.tolist(),
34                 'confidence': float(prob),
35                 'name': name,
36                 'score': float(score)
37             })
38     return faces
39
40 def load_embeddings(path):
41     embeddings = {}
42     for filename in os.listdir(path):
43         if filename.endswith(".npy"):
44             name = os.path.splitext(filename)[0]
45             data = np.load(os.path.join(path, filename))
46             embeddings[name] = data
47     print(F"Найдены эмбеддинги: {embeddings}")
48     return embeddings
49
50 def get_embedding(face_img, device, inception):
51     transform = transforms.Compose([
52         transforms.ToPILImage(),
53         transforms.Resize((160, 160)),
54         transforms.ToTensor(),
55         transforms.Normalize([0.5], [0.5])
56     ])
57
58     if face_img is None:
59         return None

```

```

60 face_tensor = transform(face_img).unsqueeze(0).to(device)
61 with torch.no_grad():
62     embedding = inception(face_tensor)[0].cpu().numpy()
63     return embedding
64
65 def recognize_face(face, embeddings, device, inception):
66     test_embedding = get_embedding(face, device, inception)
67     if test_embedding is None or test_embedding.size == 0 or not embeddings:
68         return "Лицо не обнаружено", -1
69
70     best_match = "Неизвестный"
71     best_score = -1
72
73     for person_name, person_embeddings in embeddings.items():
74         similarities = cosine_similarity(test_embedding, person_embeddings)
75         max_similarity = np.max(similarities)
76
77         if max_similarity > best_score:
78             best_score = max_similarity
79             best_match = person_name
80
81     print(best_match, best_score)
82     return best_match, best_score
83
84 def cosine_similarity(vec, ref_vecs):
85     vec = vec / np.linalg.norm(vec)
86     if ref_vecs.ndim == 1:
87         ref_vecs = ref_vecs / np.linalg.norm(ref_vecs)
88         return np.dot(vec, ref_vecs)
89     else:
90         ref_vecs = ref_vecs / np.linalg.norm(ref_vecs, axis=1, keepdims=True)
91         return np.dot(ref_vecs, vec)

```

model.py

```

1 import torch
2 import torch.nn as nn
3
4 class FingerprintEncoder(nn.Module):
5     def __init__(self):
6         import torchvision.models as models
7         from torchvision.models import resnet18, ResNet18_Weights
8
9         super(FingerprintEncoder, self).__init__()
10        base_model = models.resnet18(weights=ResNet18_Weights.DEFAULT)
11        base_model.fc = nn.Identity()
12        self.encoder = base_model
13
14    def forward(self, x):
15        return self.encoder(x)
16
17 class SiameseNetwork(nn.Module):
18    def __init__(self):
19        super(SiameseNetwork, self).__init__()
20        self.encoder = FingerprintEncoder()

```



```

21         self.fc = nn.Sequential(
22             nn.Linear(512, 256),
23             nn.ReLU(),
24             nn.Linear(256, 1),
25             nn.Sigmoid()
26         )
27
28     def forward(self, img1, img2):
29         feat1 = self.encoder(img1)
30         feat2 = self.encoder(img2)
31         diff = torch.abs(feat1 - feat2)
32         out = self.fc(diff)
33         return out
34
35     def forward_once(self, x):
36         """Метод для извлечения эмбеддингов из одного изображения"""
37         return self.encoder(x)

```

Автор ВКР

(подпись, дата)

А. В. Боев

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска